



DEPARTMENT OF COMPUTER ENGINEERING (DCE)
HWSW CODESIGN
MMH: HSCD446164

Group: 15

Name:

Họ và tên	Mã số sinh viên
Nguyễn Thành Đô	23119138
Nguyễn Tuấn Kiệt	23119157
Lê Đoàn Thái Khương	23119155
Hà Quang Huy	23119147
Nguyễn Phi Hùng	23119152

Lab 3 Designing With Vivado High Level Synthesis

Giới thiệu

Bài Lab hướng dẫn thiết kế phần cứng bằng High Level Synthesis sử dụng Vivado HLS. Sinh viên sẽ thực hiện mô phỏng, tổng hợp và tối ưu hóa thiết kế từ mã nguồn C/C++ sang RTL hardware trên FPGA.

Mục tiêu bài Lab

- + Hiểu nguyên lý High Level Synthesis
- + Làm quen Vivado HLS
- + Thực hiện C Simulation và C Synthesis
- + Tối ưu thiết kế bằng HLS Directives
- + Phân tích latency và hardware resources
- + Hiểu pipeline, unroll và array reshape
- + Phân tích interface synthesis trong FPGA

Công cụ cần thiết

Phần cứng

- + ZedBoard hoặc PynQ-Z2

Phần mềm

- + Vivado HLS 2019.2
- + Vivado Design Suite 2019.2
- + Tcl Command Prompt

Kết quả đạt được

- + Tạo project HLS bằng GUI và Tcl Script
- + Tổng hợp thiết kế từ C/C++ sang RTL
- + Chạy C Simulation và C/RTL Cosimulation

- + Tối ưu hiệu năng bằng HLS Directives
- + Phân tích latency và resource utilization
- + Hiểu nguyên lý interface synthesis trong FPGA HLS design

LÝ THUYẾT

1. High Level Synthesis

High Level Synthesis, viết tắt là HLS, là quá trình chuyển đổi chương trình viết bằng ngôn ngữ bậc cao như C/C++ thành mạch phần cứng dạng RTL Verilog/VHDL.

Bình thường muốn thiết kế phần cứng FPGA thì phải viết Verilog/VHDL. Nhưng với HLS, ta viết thuật toán bằng C/C++, ví dụ nhân ma trận, lọc ảnh, xử lý tín hiệu. Sau đó công cụ HLS sẽ phân tích code và tạo ra mạch phần cứng tương ứng.

2. Thực hiện C Simulation và C Synthesis

C Simulation là bước chạy thử code C/C++ giống như một chương trình phần mềm bình thường.

Mục đích của C Simulation là kiểm tra thuật toán có đúng không. Ví dụ với bài nhân ma trận, testbench sẽ đưa hai ma trận inA, inB vào hàm matrix_mult, sau đó kiểm tra kết quả prod có đúng không.

C Synthesis là bước công cụ HLS chuyển hàm C/C++ thành phần cứng RTL.

Sau C Synthesis, công cụ sẽ tạo report cho biết:

- Mạch chạy mất bao nhiêu clock
- Latency bao nhiêu
- Interval bao nhiêu
- Dùng bao nhiêu LUT, FF, DSP, BRAM
- Có loop nào chậm
- Có thể pipeline/unroll được không

3. Tối ưu thiết kế bằng HLS Directives

HLS Directives là các chỉ thị tối ưu mà mình đưa cho công cụ HLS để nó tạo phần cứng theo cách mình muốn.

Cùng một đoạn code C, nếu không thêm directive thì HLS có thể tạo mạch chạy chậm nhưng ít tốn tài nguyên. Nếu thêm directive, mạch có thể chạy nhanh hơn nhưng tốn nhiều phần cứng hơn.

Một số directive thường gặp:

- Pipeline: Kỹ thuật chia quá trình xử lý thành nhiều giai đoạn để các vòng lặp/dữ liệu có thể chạy chồng lên nhau, giúp tăng tốc độ xử lý.
- Unroll: Kỹ thuật trải vòng lặp thành nhiều phần cứng chạy song song để giảm thời gian tính toán nhưng tốn nhiều tài nguyên hơn.
- Array Partition: Kỹ thuật chia một mảng lớn thành nhiều phần nhỏ để có thể truy cập nhiều phần tử cùng lúc.
- Array Reshape: Kỹ thuật thay đổi cách tổ chức mảng trong phần cứng để tăng băng thông đọc/ghi dữ liệu.
- Interface: Chỉ thị dùng để xác định kiểu cổng giao tiếp của IP HLS, ví dụ AXI-Lite, AXI-Stream, RAM hoặc wire/register.

4. Phân tích latency và hardware resources

Latency là thời gian trễ của thiết kế, thường tính bằng số chu kỳ clock.

Ví dụ nếu report ghi latency là 100 cycles, nghĩa là từ lúc đưa dữ liệu vào đến lúc có kết quả đầu ra cần khoảng 100 chu kỳ clock.

Latency càng nhỏ thì thiết kế xử lý càng nhanh.

Hardware resources là tài nguyên phần cứng mà thiết kế sử dụng trên FPGA.

Các tài nguyên thường gặp:

- LUT : dùng để tạo logic tổ hợp
- FF : flip-flop, dùng để lưu trạng thái/dữ liệu
- DSP : khối nhân/cộng tốc độ cao
- BRAM : bộ nhớ RAM bên trong FPGA

5. Pipeline, unroll và array reshape

Trong thiết kế phần cứng, để đạt được hiệu năng cao, HLS cung cấp các directives cho phép can thiệp sâu vào cách dữ liệu và các phép toán được lên lịch trình. Ba kỹ thuật cốt lõi thường được sử dụng kết hợp với nhau bao gồm:

Pipeline: Đây là kỹ thuật cho phép xử lý gói đầu các lệnh. Thay vì phải chờ toàn bộ một chuỗi các phép toán của vòng lặp thứ nhất kết thúc hoàn toàn rồi mới bắt đầu vòng lặp thứ hai, Pipelining cho phép vòng lặp thứ hai bắt đầu ngay khi tài nguyên phần cứng của giai đoạn đầu tiên được giải phóng. Mục tiêu của Pipeline là giảm Initiation Interval (II) xuống mức lý tưởng (thường là $II = 1$, tức là mỗi chu kỳ xung nhịp đều có thể tiếp nhận dữ liệu mới). Sự đánh đổi ở đây là hệ thống sẽ tốn thêm tài nguyên thanh ghi (Flip-Flops) để lưu trữ các trạng thái trung gian giữa các tầng của đường ống.

Loop Unroll: Nếu Pipeline là làm việc liên tục, thì Unroll là làm nhiều việc cùng một lúc. Kỹ thuật này nhân bản trực tiếp phần thân của vòng lặp thành nhiều bản sao phần cứng vật lý hoạt động song song. Ví dụ, một vòng lặp chạy 4 lần sẽ được trải phẳng thành 4 bộ xử lý tính toán đồng thời trong một chu kỳ xung nhịp duy nhất. Việc áp dụng Unroll giúp giảm triệt để độ trễ (Latency) nhưng sẽ làm tăng vọt mức tiêu thụ tài nguyên tính toán như DSP và LUT.

Array Reshape: Khi áp dụng Pipeline và Unroll, hệ thống sẽ yêu cầu truy xuất rất nhiều dữ liệu cùng lúc. Tuy nhiên, kiến trúc bộ nhớ nhúng (BRAM) trên FPGA mặc định thường chỉ có 2 cổng (dual-port), nghĩa là chỉ cho phép tối đa 2 thao tác đọc/ghi trong một chu kỳ xung nhịp. Nếu phần cứng song song yêu cầu 3 thao tác đọc cùng lúc, hệ thống sẽ gặp hiện tượng "nút thắt cổ chai" (bottleneck). Kỹ thuật Array Reshape giải quyết vấn đề này bằng cách tổ chức lại bộ nhớ vật lý: ghép nhiều phần tử dữ liệu nhỏ thành một từ dữ liệu lớn hơn (wider word) hoặc chia mảng lớn thành nhiều khối BRAM nhỏ độc lập. Điều này giúp mở rộng băng thông bộ nhớ, đáp ứng đủ cổng truy cập cho các khối logic đang hoạt động song song.

6. Interface Synthesis trong FPGA

Một module phần cứng (IP core) trên FPGA luôn cần các chân tín hiệu vật lý để giao tiếp với các thành phần khác trong hệ thống. Interface Synthesis là quá trình biên dịch Vivado HLS tự động chuyển đổi các đối số và giá trị trả về của hàm C/C++ thành các giao thức phần cứng RTL tiêu chuẩn. Quá trình này được tổ chức và phân chia chi tiết thành hai cấp độ quản lý chính:

- Block-level Protocol:

+Để phối hợp hoạt động tổng thể của khối phần cứng, hệ thống triển khai một giao thức điều khiển trạng thái chung. Do một module xử lý (như bộ nhân ma trận) thường cần nhiều hơn một chu kỳ xung nhịp (clock cycle) để hoàn thành toàn bộ phép tính, hệ thống này mang tính chất đồng bộ. Vivado HLS sẽ tự động tổng hợp các tín hiệu cốt lõi sau:

- Tín hiệu đồng bộ (**ap_clk** và **ap_rst**): Các cổng 1-bit cung cấp xung nhịp và tín hiệu thiết lập lại (reset) đồng bộ.
- Tín hiệu điều khiển cơ chế bắt tay :Thường tuân theo tiêu chuẩn ap_ctrl_hs, bao gồm 4 tín hiệu chính: **ap_start**: Là tín hiệu đầu vào, được kích hoạt từ bên ngoài nhằm nhắc nhở khối bắt đầu thực hiện các phép toán. **ap_ready**: Tín hiệu đầu ra, báo hiệu khối đã sẵn sàng để nhận luồng dữ liệu đầu vào mới. **ap_idle**: Tín hiệu đầu ra, biểu thị rằng hệ thống đang ở trạng thái không tải hoặc đang trong quá trình xử lý dữ liệu. **ap_done**: Tín hiệu đầu ra, báo hiệu quá trình tính toán đã hoàn tất và dữ liệu ngõ ra đã có sẵn để sử dụng.

- Port-level Protocol:

+ Cách các biến số trong code C/C++ được hiện thực hóa thành các cổng (ports) và tín hiệu RTL phụ thuộc trực tiếp vào kiểu dữ liệu của chúng:

- Biến đơn trị (Scalar): Thường được phần mềm tổng hợp thành các dây dẫn tín hiệu (wires) đầu vào/ra đơn giản.
- Arrays: Cấu trúc kiểu mảng trong C++ (như các mảng đầu vào và mảng kết quả) tương ứng trực tiếp với cấu trúc của bộ nhớ vật lý, do đó phần mềm ngầm định triển khai bằng giao thức ap_memory. Một mảng khi chuyển thành RTL sẽ tự động tạo ra một tập hợp các thành phần giao diện để giao tiếp chuẩn xác với bộ nhớ BRAM bao gồm: **Data ports - d0/q0**: Dùng để nhận và truyền dữ liệu. **Address ports - address0**: Dùng để xác định vị trí dữ liệu trong bộ nhớ. Mỗi tín hiệu dữ liệu đều đi kèm với một cổng địa chỉ (ví dụ: các cổng 5-bit a_address0, b_address0, prod_address0). Tín hiệu cho phép (Enable signals): Gồm tín hiệu cho phép cấp xung nhịp/kích hoạt chip (ce0) và tín hiệu cho phép ghi (we0).

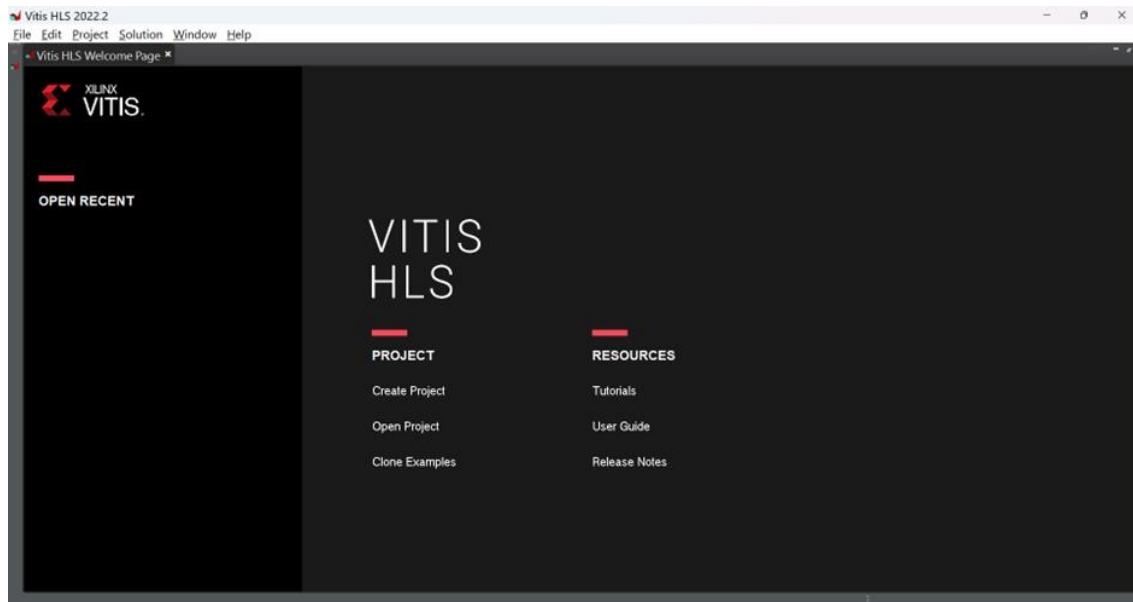
- Tác động của các Directives:

+ Cấu trúc của giao diện tổng hợp có thể bị thay đổi và chịu tác động trực tiếp bởi việc áp dụng các chỉ thị thiết kế. Cụ thể, khi sử dụng kỹ thuật phân vùng mảng (partition arrays) để chia nhỏ bộ nhớ nhằm giải quyết hiện tượng tắc nghẽn, phần mềm sẽ tự động mở rộng thêm số lượng cổng, tín hiệu điều khiển và tài nguyên phần cứng tương ứng với từng phần mảng được chia nhỏ. Cơ chế này giúp gia tăng đáng kể băng thông (bandwidth) giao tiếp của thiết kế phần cứng.

THỰC HÀNH

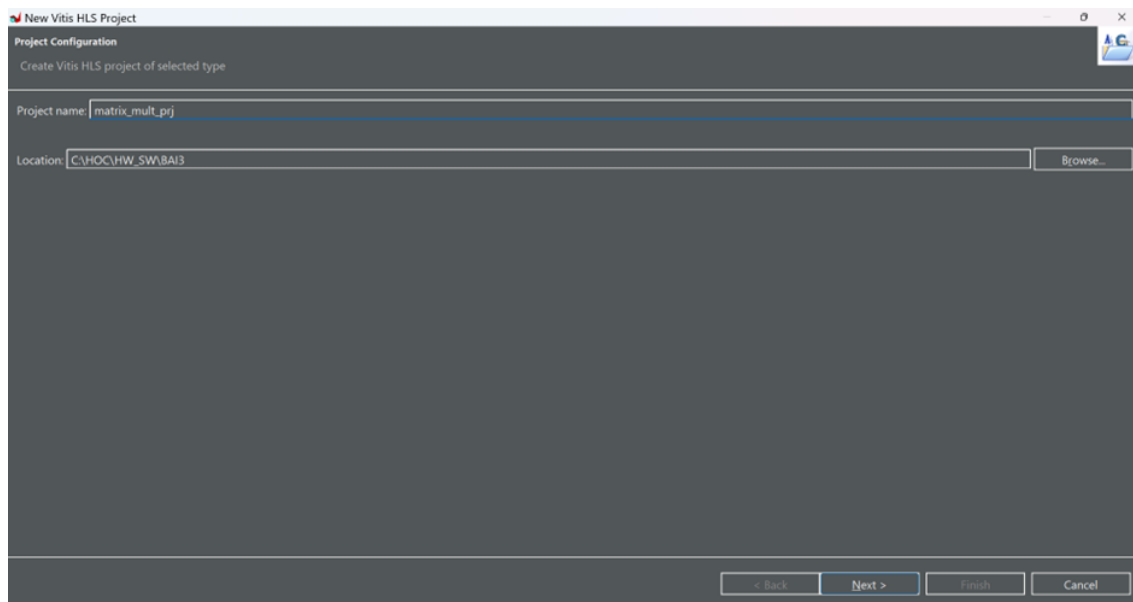
Exercise 3A - Creating Projects in Vivado HLS

Cách 1: Tạo project Vitis HLS



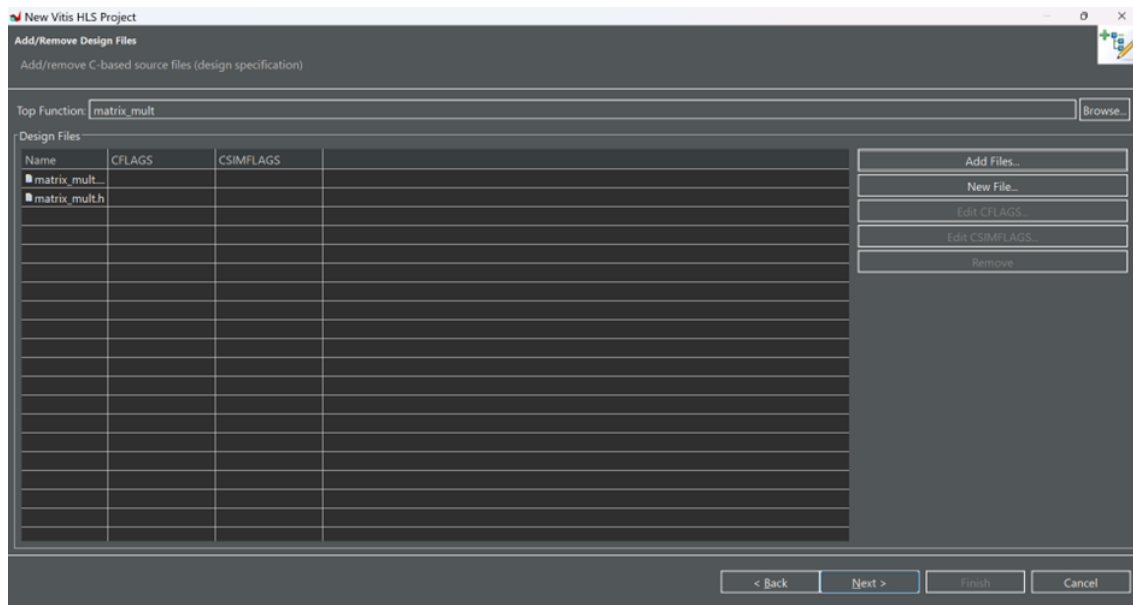
Hình 1 Giao diện chính của Vitis HLS

Ở giao diện chính của app Vitis tiến hành tạo project. Nhấn create project.



Hình 2 Setup thông tin project

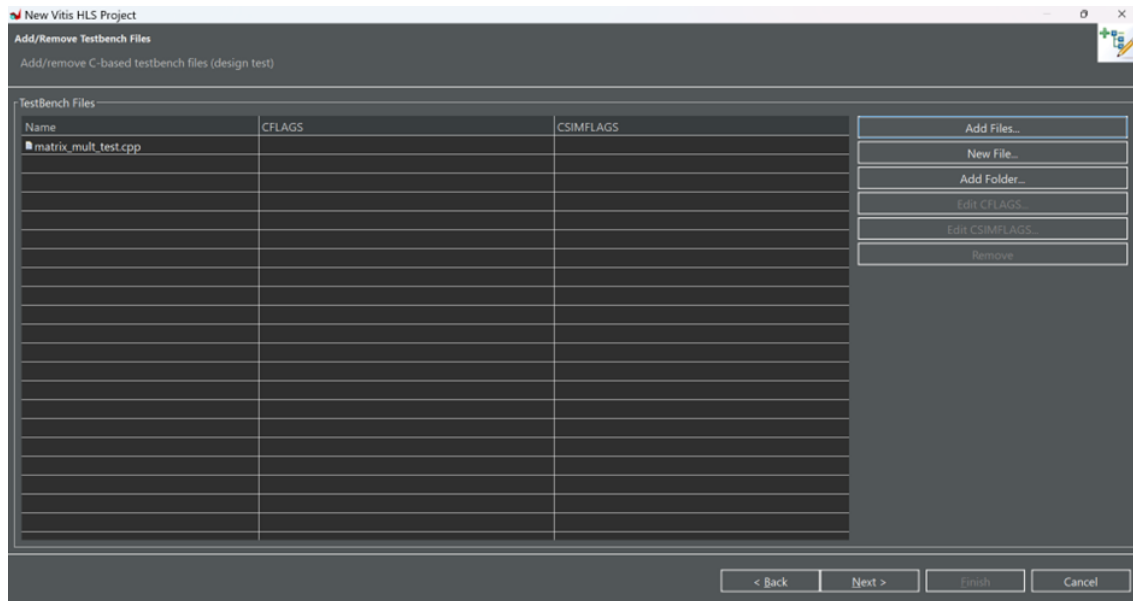
Tiến hành đặt tên và chọn đường dẫn cho project. Sau đó nhấn Next.



Hình 3 Add Design Files

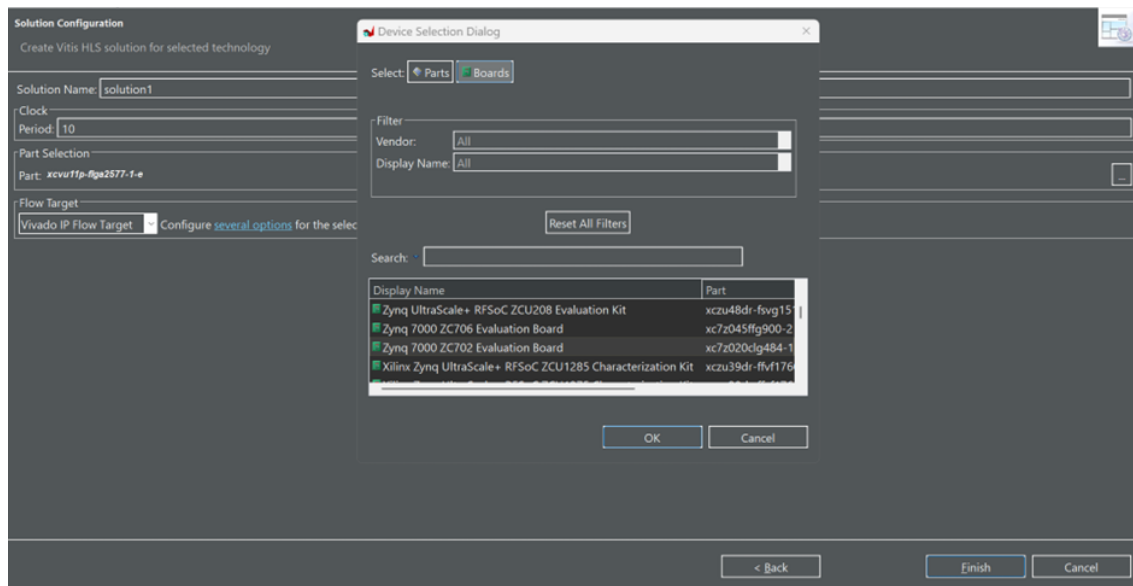
Top function là hàm chính mà Vivado HLS sẽ biến thành phần cứng.

Thêm source code `matrix_mult.cpp` và `matrix_mult.h`. Sau đó nhấn Next.



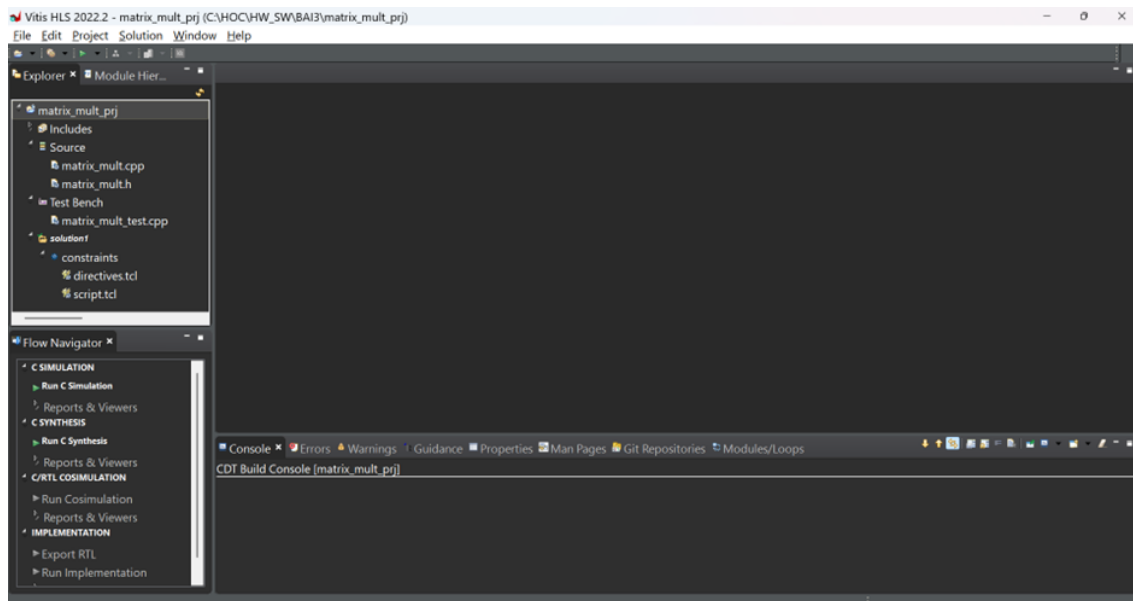
Hình 4 Add Testbench Files

Thêm source code testbench. Testbench dùng để kiểm tra xem hàm `matrix_mult` có tính đúng kết quả hay không. Sau đó nhấn Next.



Hình 5 Cấu hình Solution

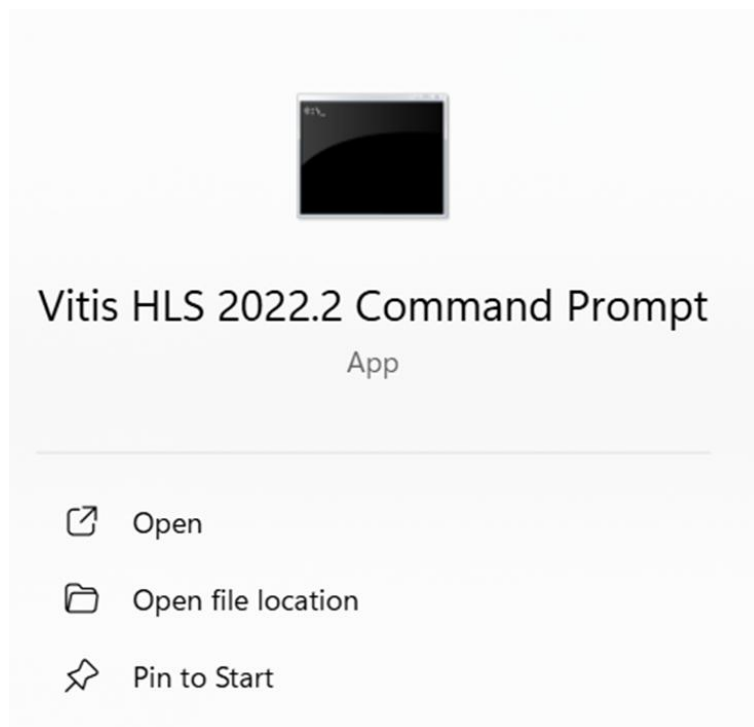
Giữ nguyên các thông số mặc định như tên là solution1, chu kỳ xung clock là 10 (đơn vị ns), và luồng target là Vivado IP. Sau đó nhấn vào dấu ba chấm ở phần Part Selection để chọn board. Chọn board như hình trên. Sau đó nhấn Finish.



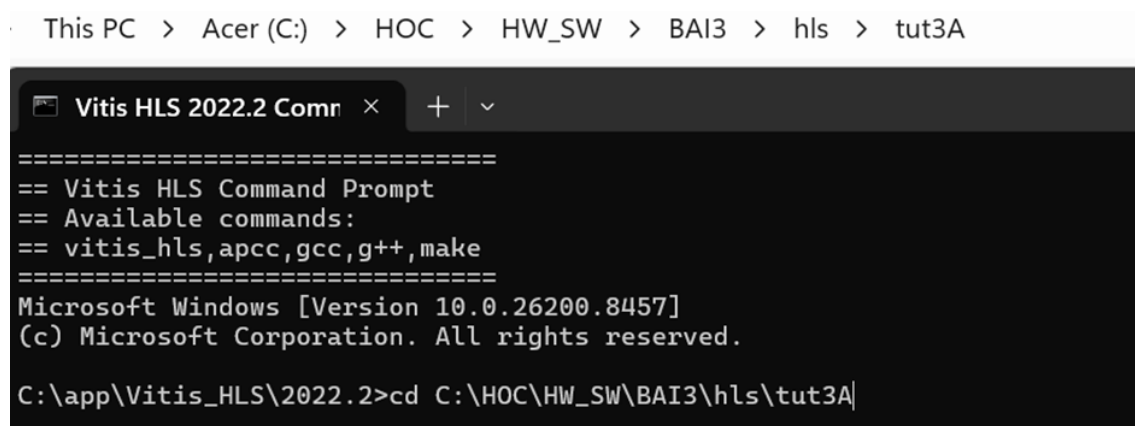
Hình 6 Giao diện sau khi tạo xong project

Cách 2: Tạo project bằng CMD.

Tại search ở window tìm kiếm hls.



Hình 7 Mở CMD của Vitis



Hình 8 Trở đến đường dẫn lưu project

Gõ lệnh như trên ảnh:

```
C:\HOC\HW_SW\BAI3\hls\tut3A>vitis_hls -f run_hls.tcl
```

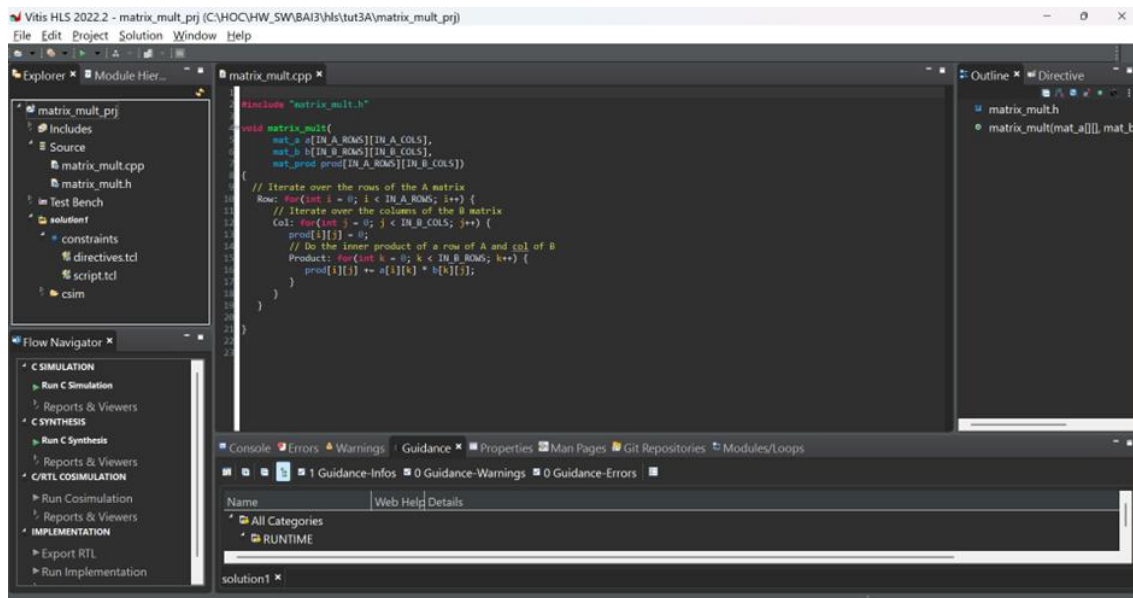
Hình 9 Lệnh tạo project

Thay vì tạo project thủ công bằng GUI, file run_hls.tcl tự động tạo project, thêm file source, thêm testbench, chọn device và chạy C simulation.

```
C:\HOC\HW_SW\BAI3\hls\tut3A>vitis_hls -p matrix_mult_prj
```

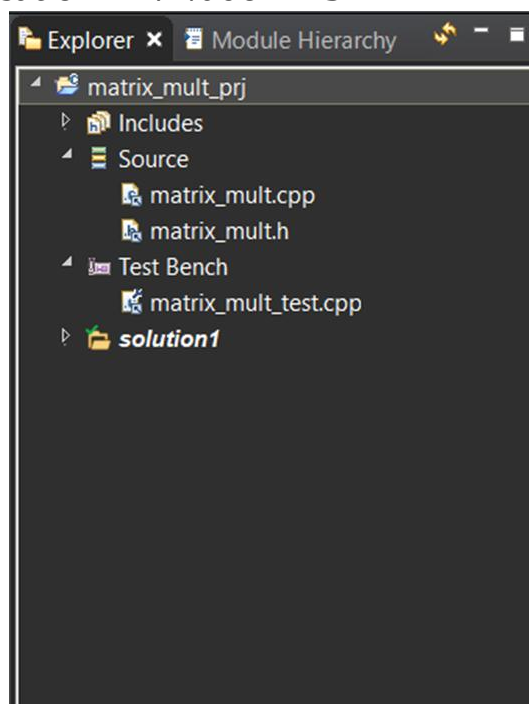
Hình 10 Lệnh mở project

Sau khi tạo xong sẽ gõ lệnh đó và mở project.

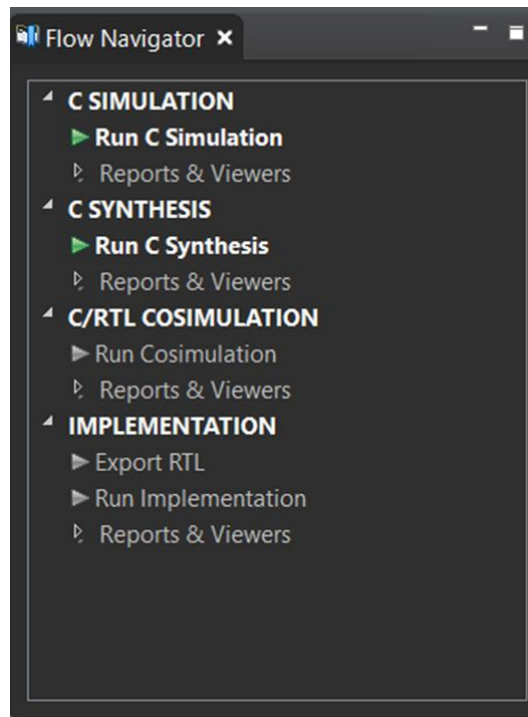


Hình 11 Giao diện sau khi tạo xong project.

Exercise 3B Design Optimisation in Vivado HLS

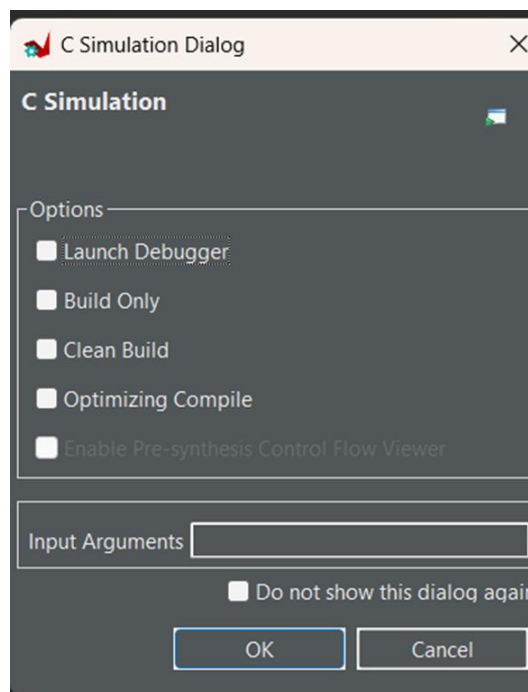


Hình 12 Thư mục project đã tạo



Hình 13 Chạy mô phỏng C solution

Nhấn chọn Run C simulation.



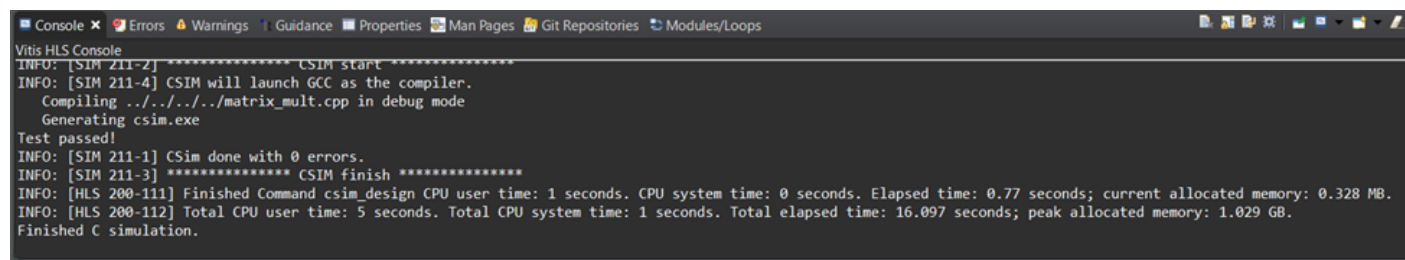
Hình 14 C Simulation Dialog trong Vitis HLS

Nó dùng để cấu hình quá trình mô phỏng và kiểm tra hoạt động của chương trình C/C++ trước khi tổng hợp sang RTL phần cứng.

- Launch Debugger: chạy mô phỏng ở chế độ debug để theo dõi từng dòng lệnh và giá trị biến.
- Build Only: chỉ biên dịch chương trình mà không chạy simulation.
- Clean Build: xóa dữ liệu build cũ và biên dịch lại từ đầu.
- Optimizing Compile: bật tối ưu hóa khi biên dịch để tăng tốc thực thi mô phỏng.

- Input Arguments: truyền tham số đầu vào cho chương trình nếu cần.

Trong bài lab này thường để mặc định và nhấn OK để chạy kiểm tra chức năng của thuật toán nhân ma trận.

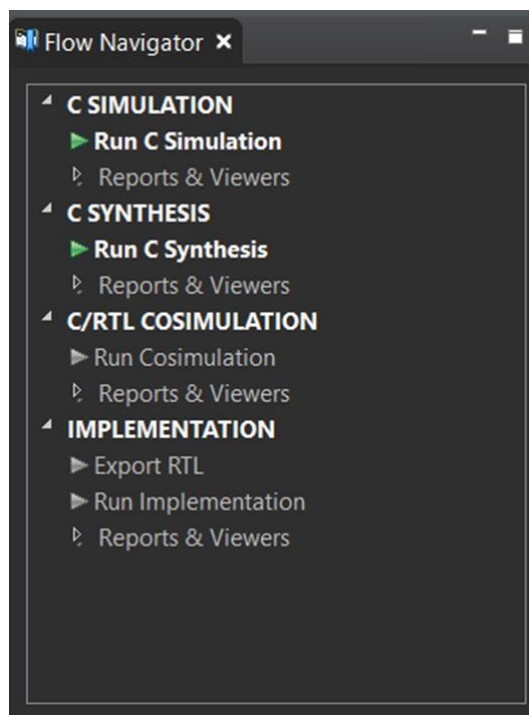


```
Vitis HLS Console
INFO: [SIM 211-2] ***** CSIM start *****
INFO: [SIM 211-4] CSIM will launch GCC as the compiler.
Compiling ../../../../matrix_mult.cpp in debug mode
Generating csim.exe
Test passed!
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] ***** CSIM finish *****
INFO: [HLS 200-111] Finished Command csim_design CPU user time: 1 seconds. CPU system time: 0 seconds. Elapsed time: 0.77 seconds; current allocated memory: 0.328 MB.
INFO: [HLS 200-112] Total CPU user time: 5 seconds. Total CPU system time: 1 seconds. Total elapsed time: 16.097 seconds; peak allocated memory: 1.029 GB.
Finished C simulation.
```

Hình 15 Console: Test passed

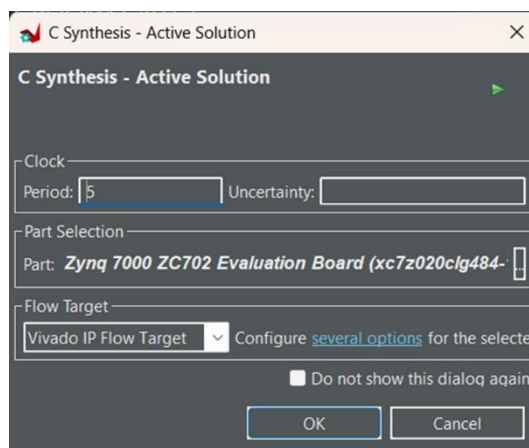
Code C/C++ chạy đúng về mặt thuật toán.

Trong phần Flow Navigator bên trái màn hình, chọn Run C Synthesis để tổng hợp IP sang file RTL.



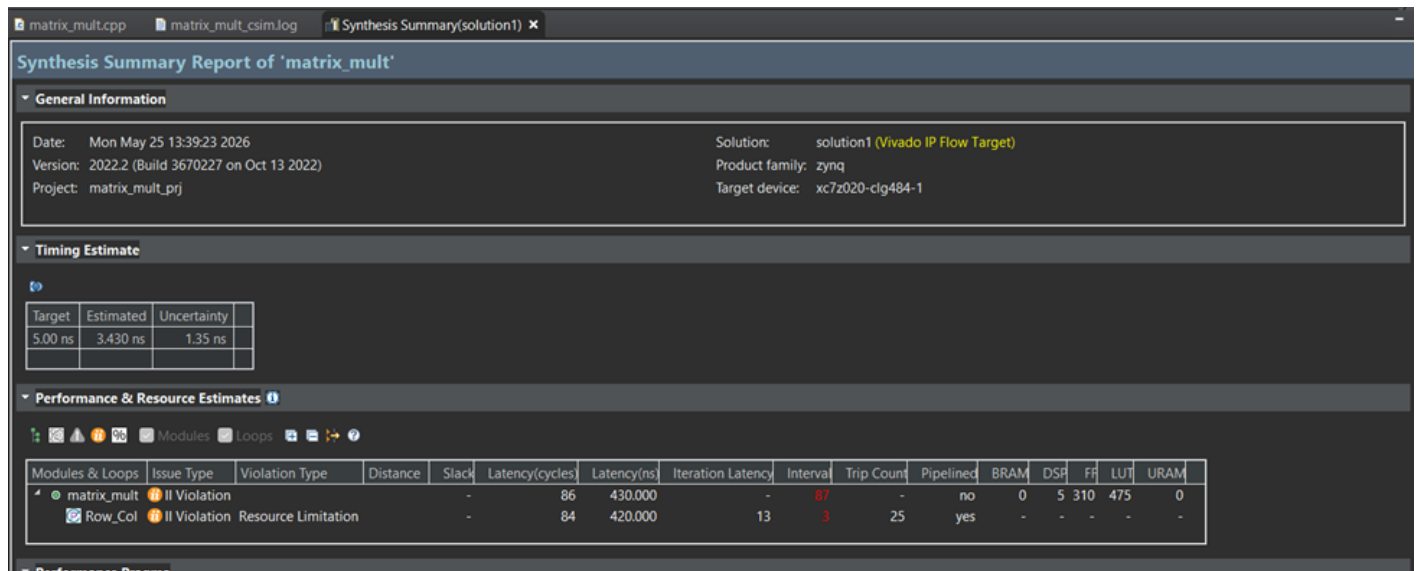
Hình 16 Run C Synthesis

Chọn cấu hình cho file synthesis. Tại đây, kiểm tra các thông số chu kì xung clock, tên board và luồng target. Sau đó nhấn OK để tiếp tục.



Hình 17 Cửa sổ cấu hình cho file synthesis

Sau khi chạy synthesis thành công. Bảng tổng hợp các thông số cơ bản của IP được thống kê như sau:



Synthesis Summary Report of 'matrix_mult'

General Information

Date:	Mon May 25 13:39:23 2026	Solution:	solution1 (Vivado IP Flow Target)
Version:	2022.2 (Build 3670227 on Oct 13 2022)	Product family:	zynq
Project:	matrix_mult_prj	Target device:	xc7z020-clg484-1

Timing Estimate

Target	Estimated	Uncertainty
5.00 ns	3.430 ns	1.35 ns

Performance & Resource Estimates

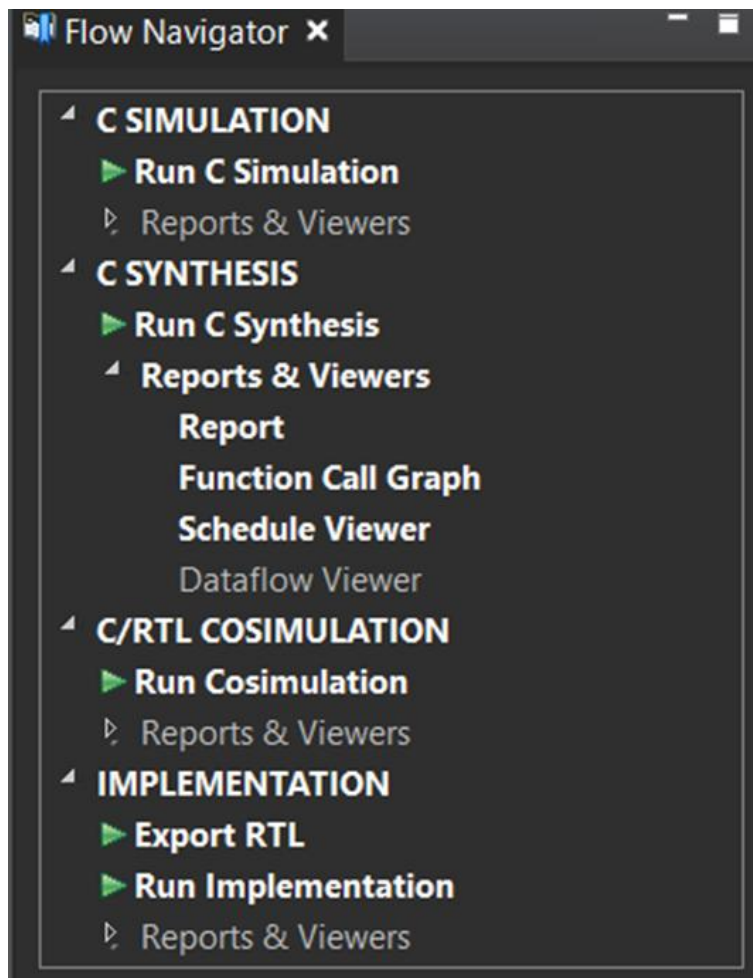
Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
matrix_mult	II Violation			-	86	430.000		87	-	no	0	5	310	475	0
Row_Col	II Violation	Resource Limitation		-	84	420.000	13	3	25	yes	-	-	-	-	-

Hình 18 Bảng tổng hợp sau synthesis

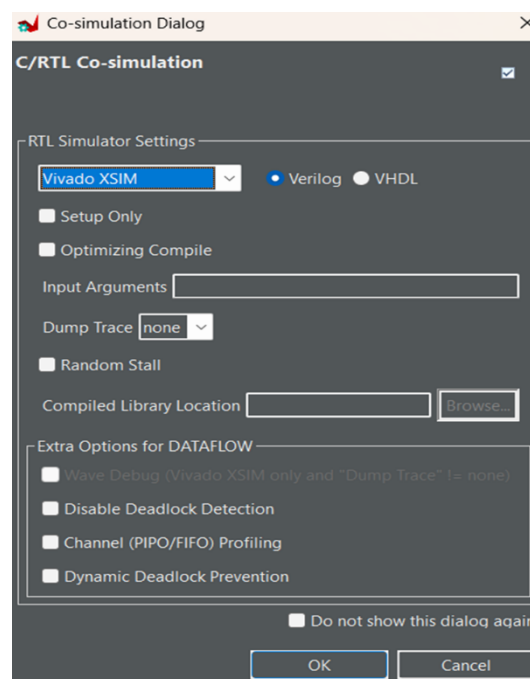
- Time Estimate (Ước lượng thời gian): Chu kỳ xung nhịp ước lượng đạt 3.430 ns, nhỏ hơn đáng kể so với giá trị mục tiêu 5.00 ns, chứng tỏ thiết kế đáp ứng rất tốt yêu cầu về thời gian (Timing) và có thể hoạt động ổn định ở tần số tối đa lên đến khoảng 291.5 Ghz.
- Độ trễ (Latency): 430 ns, tương ứng với 86 chu kỳ clock. Điều này có nghĩa là cần tiêu tốn 86 chu kỳ hoạt động tính từ khi dữ liệu đầu vào bắt đầu được đưa vào hệ thống cho đến khi hoàn -thành tính toán và xuất ra toàn bộ kết quả.
- Khoảng cách khởi tạo (Interval - II): Khoảng cách khởi tạo của vòng lặp Row_Col là 3 chu kỳ, tức là sau mỗi 3 chu kỳ clock, hệ thống mới có thể nạp dữ liệu để bắt đầu xử lý một phép tính mới trong pipeline. Hệ thống báo lỗi màu đỏ II Violation do Resource Limitation, chứng tỏ công cụ đã cố gắng tối ưu để đạt II=1 nhưng thất bại và phải dẫn ra thành II=3 do mạch bị giới hạn tài nguyên, thường là do thiếu cổng đọc/ghi của bộ nhớ.
- Resource Utilization (Sử dụng tài nguyên): Thiết kế sử dụng 0 BRAM, 5 DSP, 310 FF và 475 LUT, cho thấy tài nguyên phần cứng được khai thác ở mức rất thấp và cực kỳ tiết kiệm diện tích. Trong đó, DSP thực hiện các phép toán nhân ma trận, FF dùng để lưu trữ dữ liệu 1-bit qua các nhịp clock, LUT thực hiện các phép toán logic (như rẽ nhánh, đếm vòng lặp), và hệ thống chưa cần dùng đến bộ nhớ tốc độ cao BRAM.

Thực hiện Cosimulation với mã C/RTL

Trong phần Flow Navigator, chọn Run Cosimulation để kiểm tra mã RTL bằng code testbench C/C++ ban đầu.



Hình 19 Run Cosimulation



Hình 20 C/RTL Co-simulation Dialog trong Vitis HLS

Hộp thoại này dùng để cấu hình quá trình C/RTL Co-simulation trong Vitis HLS. Đây là bước kiểm tra xem RTL (Verilog/VHDL) được sinh ra từ HLS có hoạt động giống với chương trình C/C++ ban đầu hay không.

Cosimulation Report for 'matrix_mult'

General Information

Date: Mon May 25 13:44:11 SEAST 2026
Version: 2022.2 (Build 3670227 on Oct 13 2022)
Project: matrix_mult_prj
Status: Pass

Solution: solution1 (Vivado IP Flow Target)
Product family: zynq
Target device: xc7z020-clg484-1

Cosim Options

Tool: Vivado XSIM
RTL: Verilog

Performance Estimates

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
matrix_mult				84	84	84
Row_Col				85	85	85

Hình 21 Kết quả sau khi Cosimulation

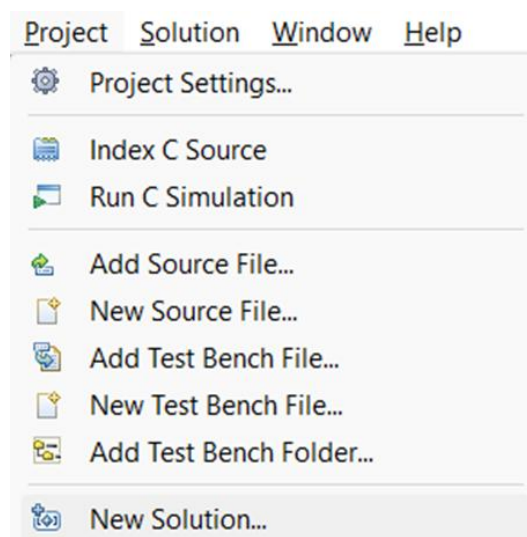
Kết quả C/RTL Co-simulation của là pass và cho thấy RTL Verilog được sinh ra từ quá trình HLS hoạt động chính xác và cho kết quả giống với chương trình C/C++ ban đầu.

Latency: 84 cycles

Latency là thời gian mạch cần để hoàn thành một lần xử lý. Ở đây mạch mất 84 chu kỳ clock để thực hiện phép nhân ma trận.

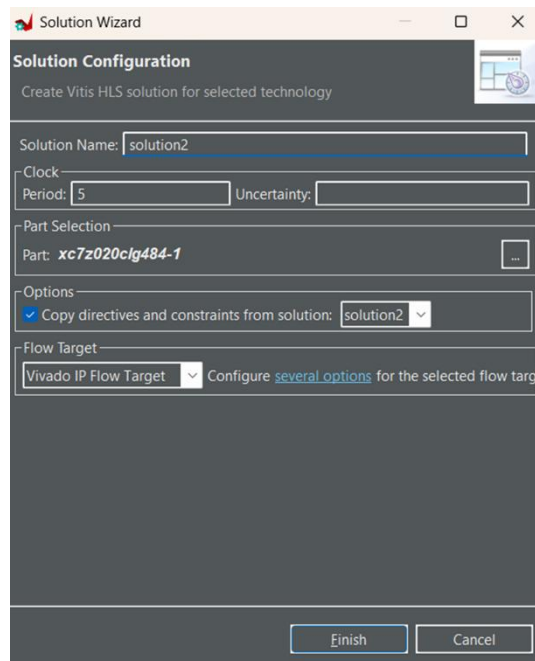
Row_Col latency: 85 cycles, gần bằng giá trị 86 chu kỳ đo được ở bước Synthesis. Cho thấy được quá trình tạo mã RTL thành công.

Row_Col là vòng lặp chính của thuật toán nhân ma trận. Độ trễ 85 chu kỳ cho thấy phần lớn thời gian xử lý nằm ở vòng lặp này.



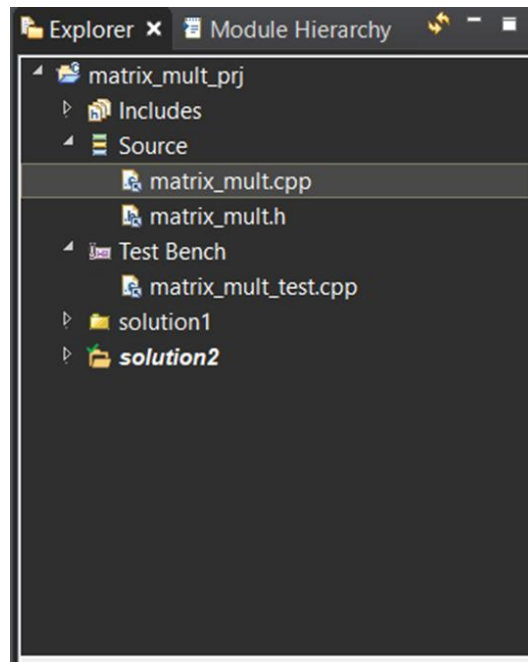
Hình 22 Tạo Solution mới

Chọn Project → New Solution

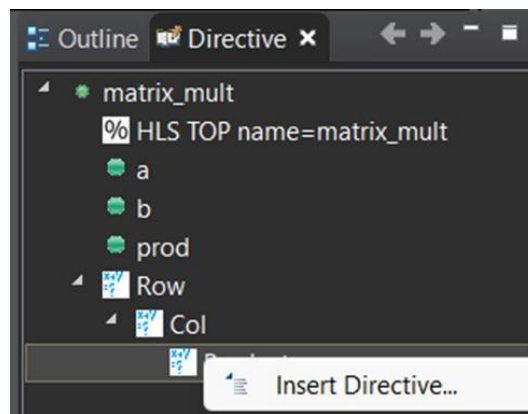


Hình 23 Cửa sổ cấu hình solution2

Xuất hiện cửa sổ cấu hình cho solution2, để mặc định và bấm Finish

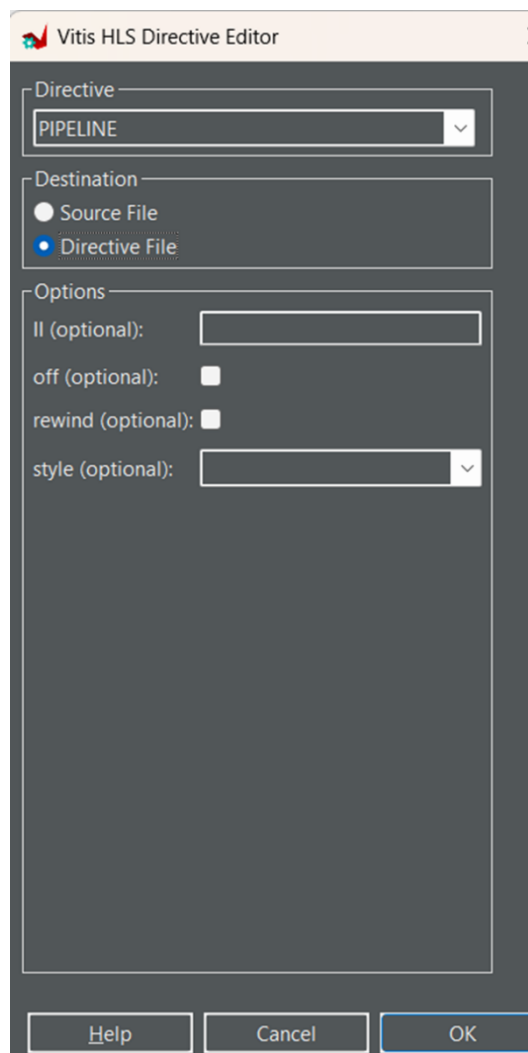


Hình 24 Tab Explorer sau khi tạo solution2



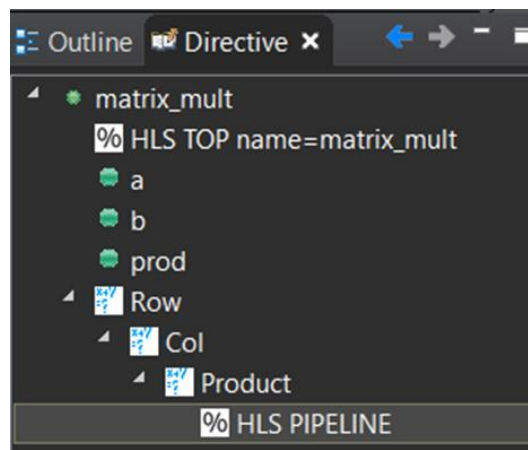
Hình 25 Tab Directive của solution2

Chọn tab Directive ở phía bên phải màn hình, nhấn chuột phải vào mục Product → Insert Directive sẽ xuất hiện cửa sổ lựa chọn cách thức tối ưu phần cứng



Hình 26 Cửa sổ cấu hình các cách tối ưu phần cứng cho solution2

Tại mục Direction, chọn PIPELINE và nhấn OK



Hình 27 Tab Directive sau khi thêm PIPELINE

Timing Estimate

Target	Estimated	Uncertainty
5.00 ns	3.610 ns	1.35 ns

Performance & Resource Estimates

Hình 28 Tab synthesis summary của solution2

- Time Estimate (Ước lượng thời gian): Chu kỳ xung nhịp ước lượng đạt 3.610 ns, nhỏ hơn so với chu kỳ mục tiêu 5.00 ns, chứng tỏ thiết kế đáp ứng yêu cầu về timing. Với chu kỳ 3.610 ns, thiết kế về mặt lý thuyết có thể đạt tần số khoảng 277 Ghz, cao hơn tần số mục tiêu 200 Ghz. Điều này cho thấy mạch sau C Synthesis có khả năng hoạt động ổn định ở mức timing yêu cầu.

- Độ trễ (Latency): Toàn bộ hàm matrix_mult có độ trễ 258 chu kỳ clock, tương ứng khoảng 1290 ns. Điều này có nghĩa là từ khi dữ liệu đầu vào bắt đầu được đưa vào hệ thống cho đến khi hoàn thành phép nhân ma trận và tạo ra kết quả đầu ra, thiết kế cần khoảng 258 chu kỳ clock.

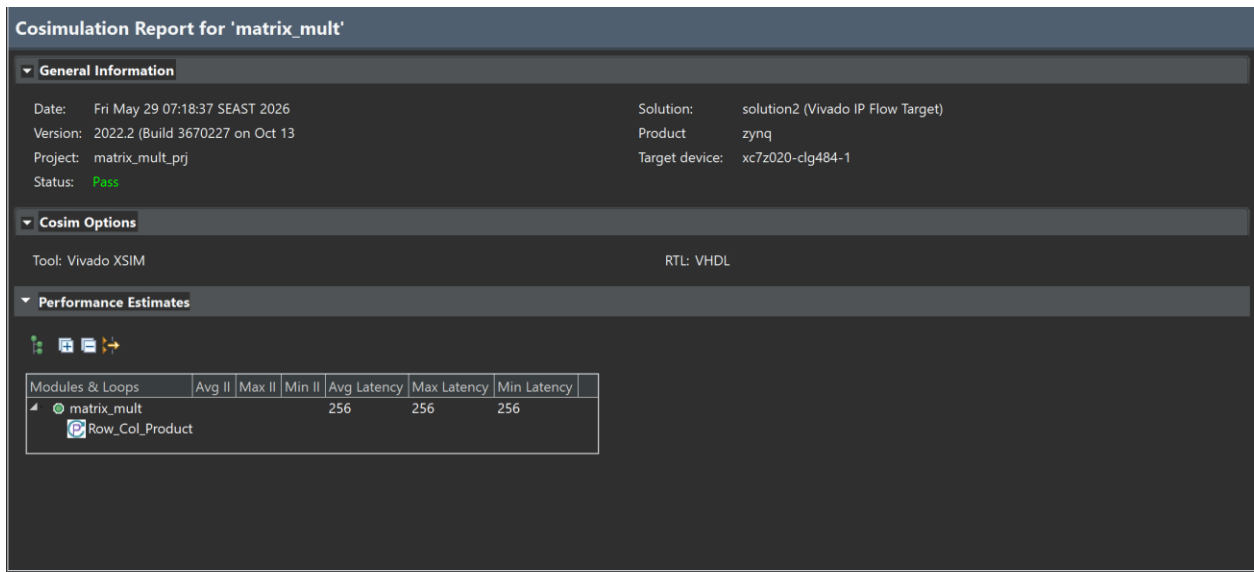
- Khoảng cách khởi tạo (Interval - II): Hàm matrix_mult có interval là 259 chu kỳ, nghĩa là sau khi bắt đầu một lần tính toán, hệ thống phải chờ khoảng 259 chu kỳ mới có thể bắt đầu xử lý bộ dữ liệu tiếp theo. Riêng vòng lặp Row_Col_Product đã được pipeline và có II = 2, tức là sau mỗi 2 chu kỳ clock, pipeline mới có thể bắt đầu một vòng lặp mới. Hệ thống xuất hiện cảnh báo màu cam II Violation do Memory Dependency, cho thấy công cụ HLS đã cố gắng tối ưu để đạt II nhỏ hơn nhưng không thể đạt II = 1 do tồn tại phụ thuộc dữ liệu/bộ nhớ trong quá trình tính toán, thường liên quan đến thao tác đọc, cập nhật và ghi lại giá trị trung gian của ma trận kết quả.

- Resource Utilization (Sử dụng tài nguyên): Thiết kế sử dụng 0 BRAM, 1 DSP, 385 FF và 507 LUT, cho thấy tài nguyên phần cứng được sử dụng ở mức khá thấp. Việc chỉ dùng 1 DSP cho thấy phép nhân trong ma trận chưa được song song hóa mạnh, giúp tiết kiệm tài nguyên nhưng làm hiệu năng chưa tối đa. FF được dùng để lưu trữ dữ liệu và trạng thái qua các chu kỳ clock, còn LUT dùng để thực hiện logic điều khiển và các phép toán tổ hợp. Thiết kế không dùng BRAM, cho thấy dữ liệu mảng trong bài toán này chưa cần ánh xạ vào bộ nhớ khối lớn của FPGA.

- Nhận xét tổng quát: Solution2 đã áp dụng pipeline cho vòng lặp Row_Col_Product, giúp cải thiện khả năng xử lý so với thiết kế tuần tự hoàn toàn. Tuy nhiên, do xuất hiện Memory Dependency, pipeline chỉ đạt II = 2 thay vì II = 1. Thiết kế đạt timing và sử dụng ít tài nguyên, đặc biệt chỉ dùng 1 DSP, nhưng hiệu năng vẫn bị

giới hạn

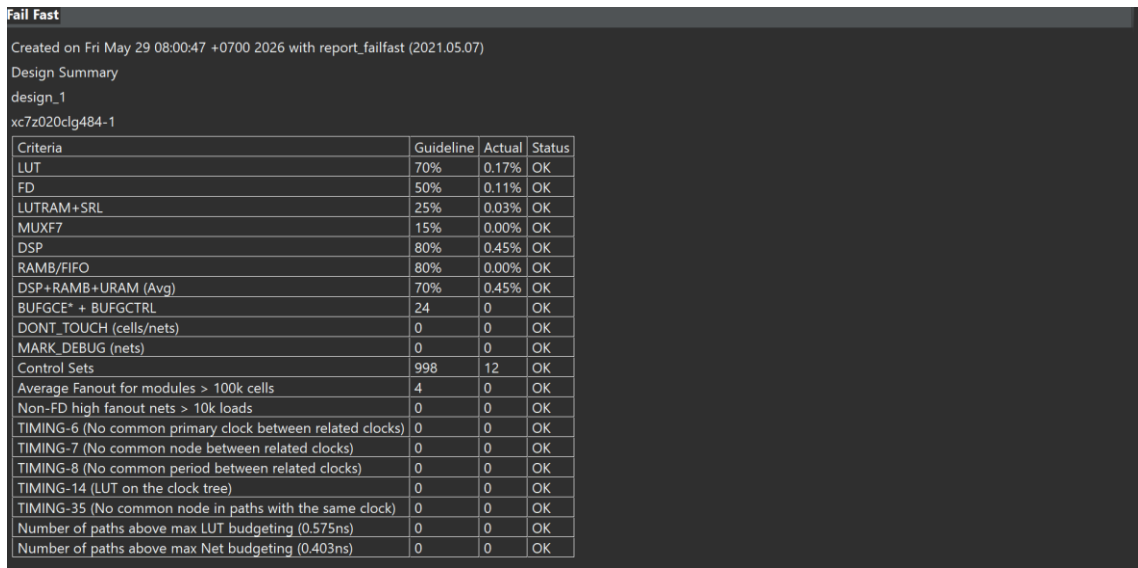
bởi phụ thuộc dữ liệu và khả năng truy cập bộ nhớ. Vì vậy, solution2 phù hợp khi muốn tiết kiệm tài nguyên phần cứng, nhưng chưa phải là phương án tối ưu nhất về tốc độ xử lý.



Hình 29 Kết quả cosimulation solution 2

Trạng thái báo Pass. Điều này chứng tỏ RTL được tạo ra từ HLS đã mô phỏng thành công và cho kết quả đúng so với testbench.

Performance Estimates (Ước lượng hiệu năng): Hàm matrix_mult có Avg Latency = 256, Max Latency = 256 và Min Latency = 256. Điều này cho thấy mỗi lần thực hiện phép nhân ma trận trong mô phỏng RTL đều cần đúng 256 chu kỳ clock, không bị dao động giữa các lần chạy.



Hình 30 Kết quả implementation của solution 2

- Tổng quan: Tất cả các tiêu chí trong cột Status đều là OK, nghĩa là thiết kế không vi phạm các giới hạn kiểm tra nhanh. Không có lỗi nghiêm trọng về tài nguyên, fanout, clock, net delay hoặc timing path.
- Resource Utilization: Thiết kế sử dụng tài nguyên rất thấp so với giới hạn của FPGA. LUT chỉ dùng 0.17%, FD/FF dùng 0.11%, DSP dùng 0.45%, LUTRAM + SRL dùng 0.03%, còn RAMB/FIFO = 0.00%. Điều này cho thấy mạch rất nhỏ so với khả năng của chip Zynq-7020, tài nguyên FPGA còn dư rất nhiều.
- DSP Usage: DSP có guideline là 80%, còn thực tế chỉ dùng 0.45%, trạng thái OK. Điều này cho thấy các phép toán

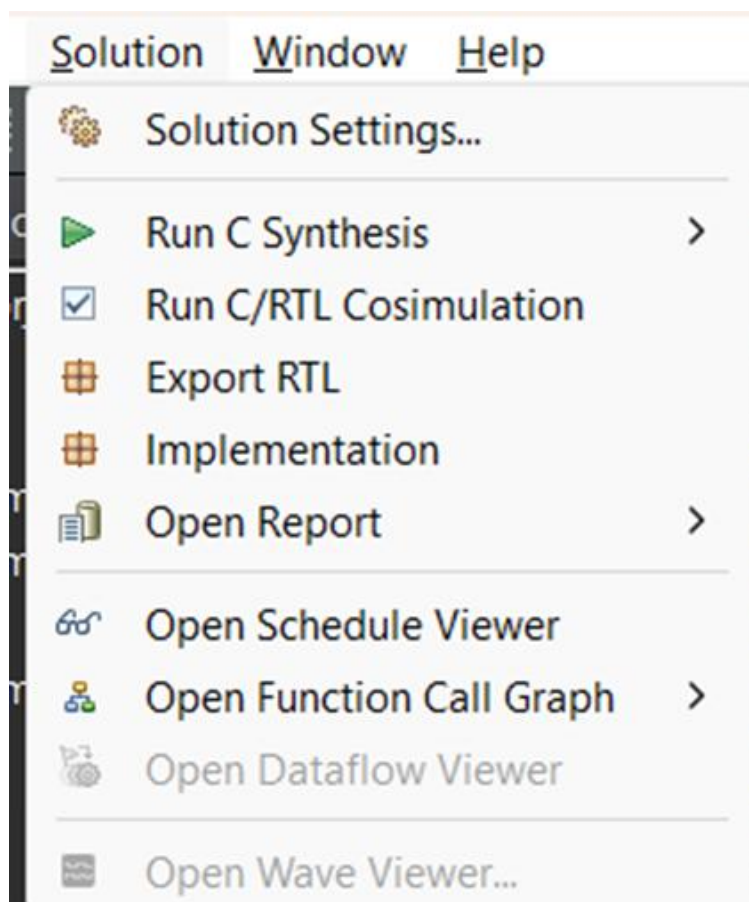
nhân trong thiết kế chỉ dùng một lượng rất nhỏ khối DSP, không gây áp lực lên tài nguyên tính toán chuyên dụng của FPGA.

- Control Sets: Số lượng Control Sets thực tế là 12, nhỏ hơn rất nhiều so với guideline 998, nên trạng thái OK. Control set liên quan đến các tín hiệu điều khiển như clock enable, reset và set/reset của flip-flop. Giá trị thấp cho thấy thiết kế không phức tạp về mặt điều khiển.

- Fanout: Mục Average Fanout for modules > 100k cells có giá trị thực tế là 0, trạng thái OK. Điều này cho thấy thiết kế không có module cực lớn hoặc tín hiệu điều khiển bị phân nhánh quá nhiều, nên không gây áp lực lớn lên routing.

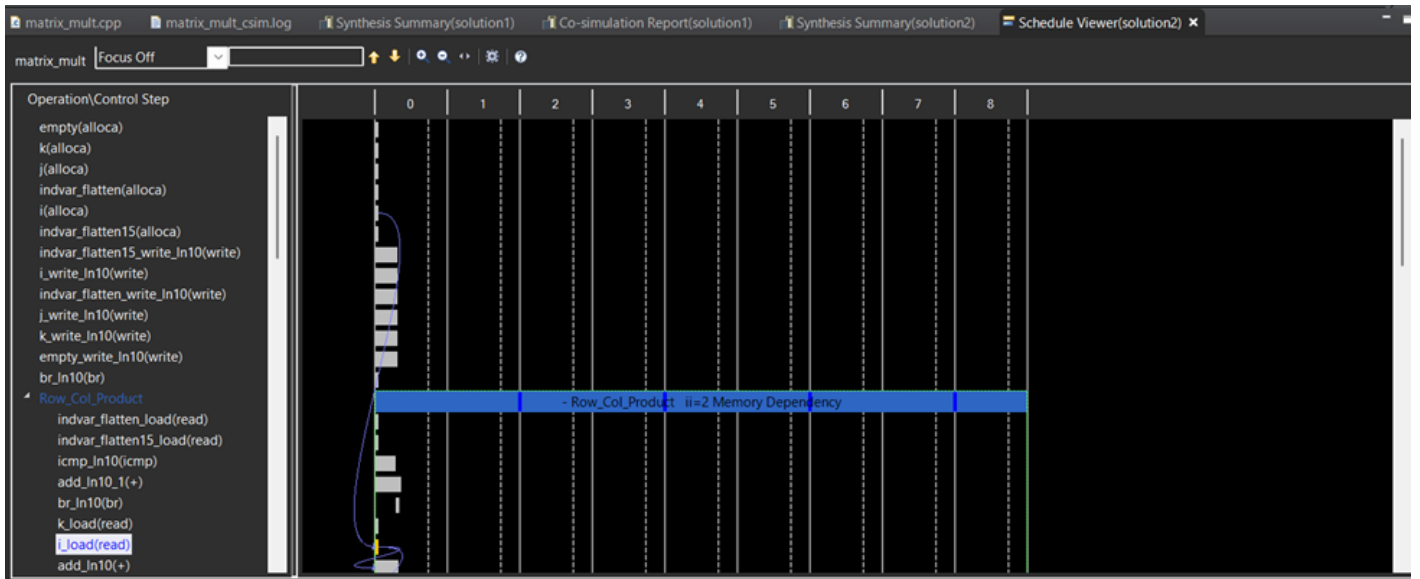
- Timing Checks: Các mục TIMING-6, TIMING-7, TIMING-8, TIMING-14, TIMING-35 đều có giá trị 0 và trạng thái OK. Điều này có nghĩa là không phát hiện lỗi liên quan đến clock domain, clock tree, hoặc các đường timing bất thường giữa các clock liên quan.

- Net Delay Budgeting: Hai mục Number of paths above max LUT budgeting và Number of paths above max Net budgeting đều bằng 0, trạng thái OK. Điều này cho thấy không có đường tín hiệu nào có độ trễ logic hoặc độ trễ dây nối vượt quá giới hạn cho phép.



Hình 31 Tab mở Schdule Viewer

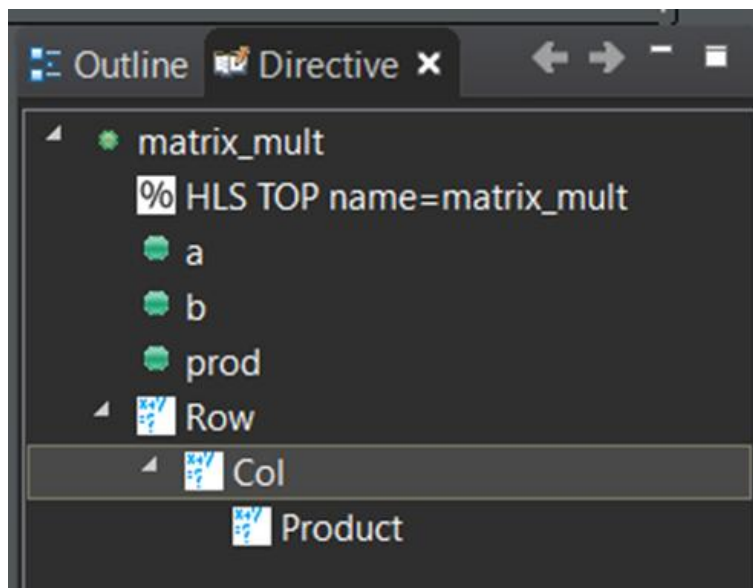
Chọn Solution → Open Schedule Viewer để biết cách trình biên dịch chuyển đổi thuật toán thành quá trình xử lý của phần cứng



Hình 32 Cửa sổ Schedule Viewer

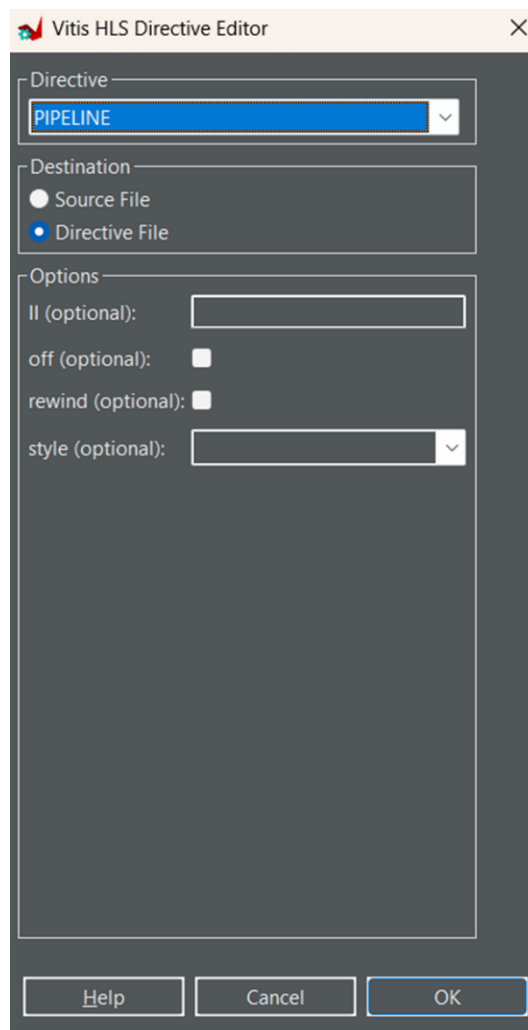
Hình 33 Cửa sổ tạo solution3

Tạo Solution3 như solution 2, nhưng ở mục options thì chọn solution1, nếu để mặc định sẽ là solution2, và bấm Finish



Hình 34 Tab Directive của solution3

Nhấn chuột phải vào mục Col → insert Directive → chọn PIPELINE



Hình 35 Cửa sổ cấu hình các cách tối ưu phần cứng cho solution3

Run C Synthesis, tại ô bên cạnh matrix_mult chọn II Violation



Hình 36 Cửa sổ synthesis cho II violation

Timing Estimate

Target	Estimated	Uncertainty
10.00 ns	5.932 ns	2.70 ns

Performance & Resource Estimates

Modules

Loops

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT
matrix_mult				-	131	1.310E3	-	132	-	no	0	1	317	443
☒ Row_Col_Product				-	129	1.290E3	6	1	125	yes	-	-	-	-

Hình 37 Synthesis report solution 3

- Time Estimate: Chu kỳ xung nhịp ước lượng đạt khoảng 5.932 ns, nhỏ hơn mục tiêu 10.00 ns, nên thiết kế vẫn đáp ứng timing. Tần số tối đa xấp xỉ 168.6 Ghz.
- Latency: Thiết kế có độ trễ khoảng 129 chu kỳ, cao hơn so với solution basic. Điều này nghĩa là từ lúc nhận dữ liệu đầu vào đến khi hoàn thành phép nhân ma trận cần khoảng 129 chu kỳ clock.
- Interval / II: Vòng lặp chính có Interval = 32, tức là sau 32 chu kỳ mới có thể bắt đầu xử lý lần tính tiếp theo. So với basic, II lớn hơn nên khả năng pipeline, kế tiếp dữ liệu chưa tốt bằng.
- Resource Utilization: Thiết kế dùng 1 DSP, 57 FF, 97 LUT, 0 BRAM, cho thấy tài nguyên phần cứng giảm rất mạnh, đặc biệt DSP giảm từ 5 xuống còn 1.

Cosimulation Report for 'matrix_mult'

General Information

Date: Fri May 29 00:31:48 SEAST 2026

Version: 2022.2 (Build 3670227 on Oct 13 2022)

Project: matrix_mult_prj

Status: Pass

Solution: solution2 (Vivado IP Flow Target)

Product family: zynq

Target device: xc7z020-clg484-1

Cosim Options

Tool: Vivado XSIM

RTL: Verilog

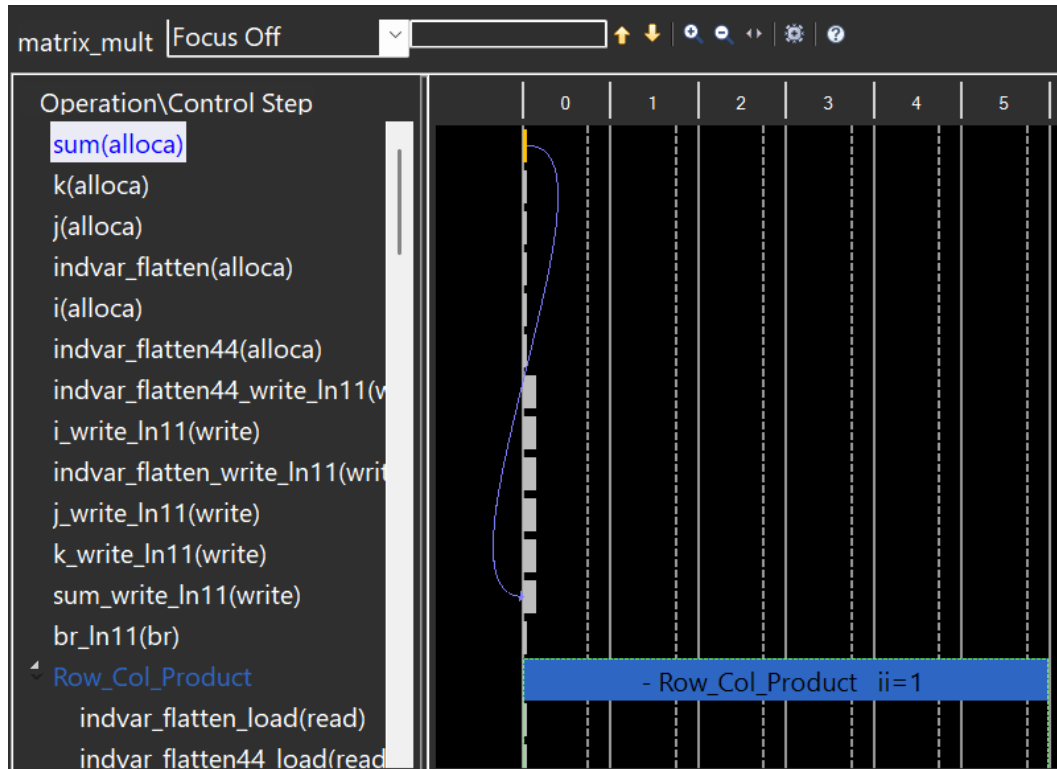
Performance Estimates

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
matrix_mult				129	129	129
Row_Col_Product				130	130	130

Hình 38 Cosimulation report solution 3

Kết quả Co-simulation cho thấy thiết kế chạy đúng chức năng.

Latency thực tế khoảng 129–130 chu kỳ, gần với kết quả synthesis, chứng tỏ RTL được sinh ra đúng với ước lượng ban đầu.



Hình 39 Schedule Viewer solution 3

Schedule Viewer biểu diễn cách Vivado HLS sắp xếp các phép toán theo từng Control Step (chu kỳ clock). Trục ngang thể hiện số chu kỳ thực hiện, còn bên trái là các biến và thao tác được cấp phát tài nguyên.

Trong hình, vòng lặp `Row_Col_Product` được thực hiện liên tục từ khoảng chu kỳ 0 đến chu kỳ 5, cho thấy đây là khối xử lý chính của phép nhân ma trận. Thanh màu xanh kéo dài qua nhiều chu kỳ chứng tỏ phép tính tích lũy (multiply-accumulate) cần nhiều bước xử lý trước khi hoàn thành.

Các biến như `i`, `j`, `k`, `sum` được cấp phát và ghi dữ liệu ở giai đoạn đầu của quá trình thực hiện. Sau đó dữ liệu được đưa vào khối `Row_Col_Product` để thực hiện phép nhân và cộng tích lũy.

Kết hợp với báo cáo Resource cho thấy phép nhân (`mul`) và cộng (`add`) đều được ánh xạ vào DSP48 với latency 3 chu kỳ, nên bộ lập lịch phải phân bổ các phép toán qua nhiều control step thay vì thực hiện đồng thời. Điều này giải thích vì sao thiết kế chỉ sử dụng 1 DSP nhưng có latency cao hơn so với solution basic.

Nhìn chung, Schedule Viewer cho thấy thiết kế ưu tiên tiết kiệm tài nguyên phần cứng bằng cách tái sử dụng cùng một khối DSP cho nhiều phép toán liên tiếp. Đổi lại, thời gian thực hiện của mỗi phép nhân ma trận tăng lên, dẫn đến latency tổng thể lớn hơn.

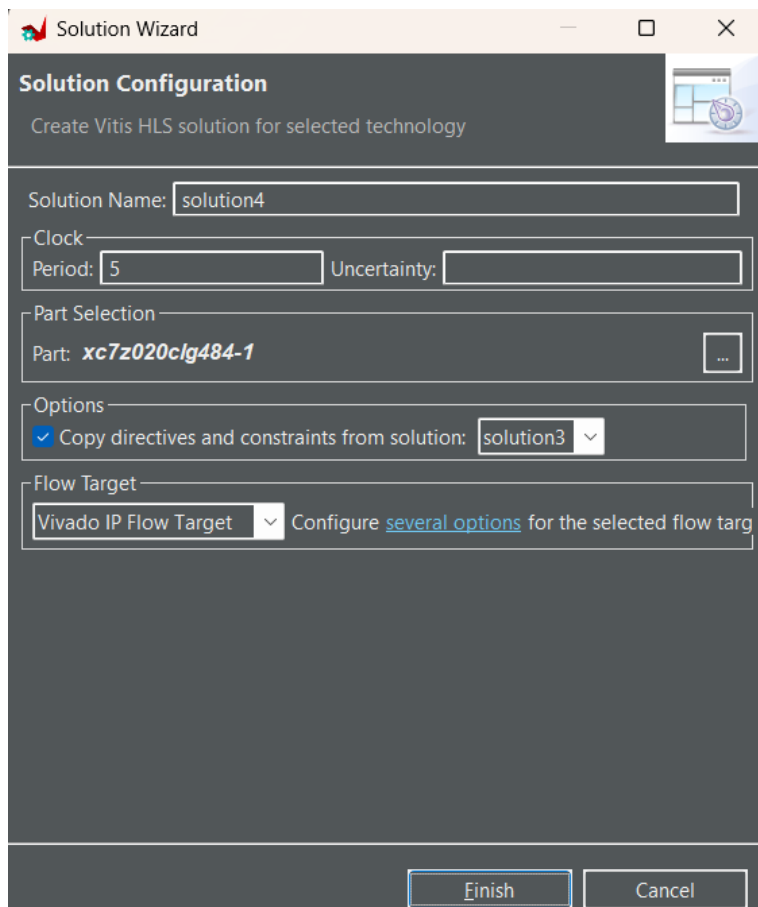
So sánh `Matrix_mul` basic và `Matrix_mul` array_partition:

Thông số	Matrix_mul basic	Matrix_mul array_partition	Nhận xét
Target clock	5 ns	10 ns	Hai solution đặt mục tiêu clock khác nhau
Estimated clock	3.430 ns	5.932 ns	Basic nhanh hơn về chu kỳ clock
Latency synthesis	86 cycles	129 cycles	Basic có latency thấp hơn
Latency cosim	85 cycles	129–130 cycles	Basic chạy nhanh hơn
Interval / II	3	32	Basic có khả năng khởi tạo tác vụ mới tốt hơn
DSP	5	1	Array_partition tiết kiệm DSP hơn
FF	310 synthesis / 161 RTL	57 RTL	Array_partition dùng FF ít hơn
LUT	475 synthesis / 104 RTL	97 RTL	Array_partition dùng LUT hơi ít hơn
BRAM	0	0	Cả hai không dùng BRAM

Nhận xét:

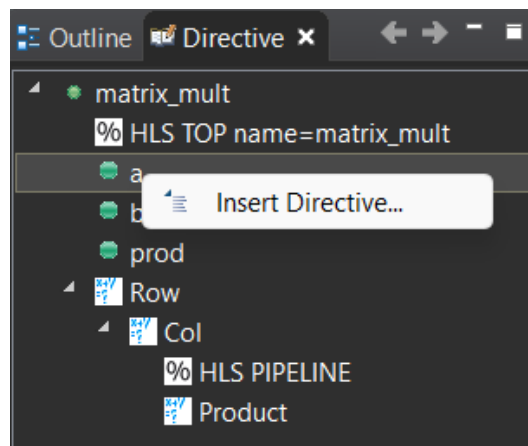
Matrix_mul basic cho hiệu năng tốt hơn vì latency thấp hơn, II nhỏ hơn và clock đạt nhanh hơn. Tuy nhiên, basic dùng nhiều DSP hơn.

Matrix_mul dùng array_partition tiết kiệm tài nguyên hơn, đặc biệt DSP giảm còn 1 DSP, FF và LUT cũng thấp hơn. Tuy nhiên, đổi lại latency tăng lên khoảng 129–130 chu kỳ và Interval tăng lên 32, nên tốc độ xử lý tổng thể kém hơn basic.



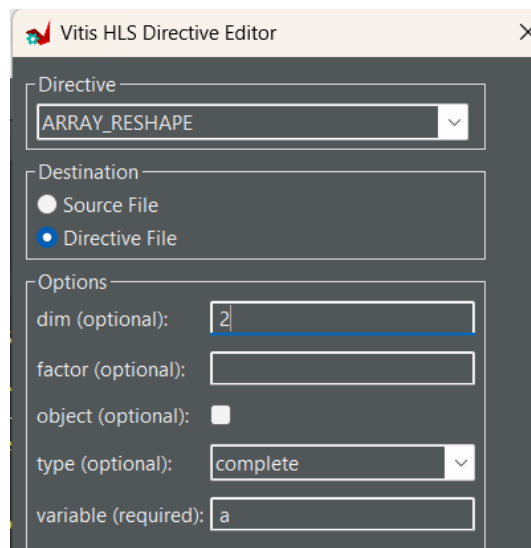
Hình 40 Cửa sổ tạo solution4

Tạo như solution trước, để mặc định và bấm Finish

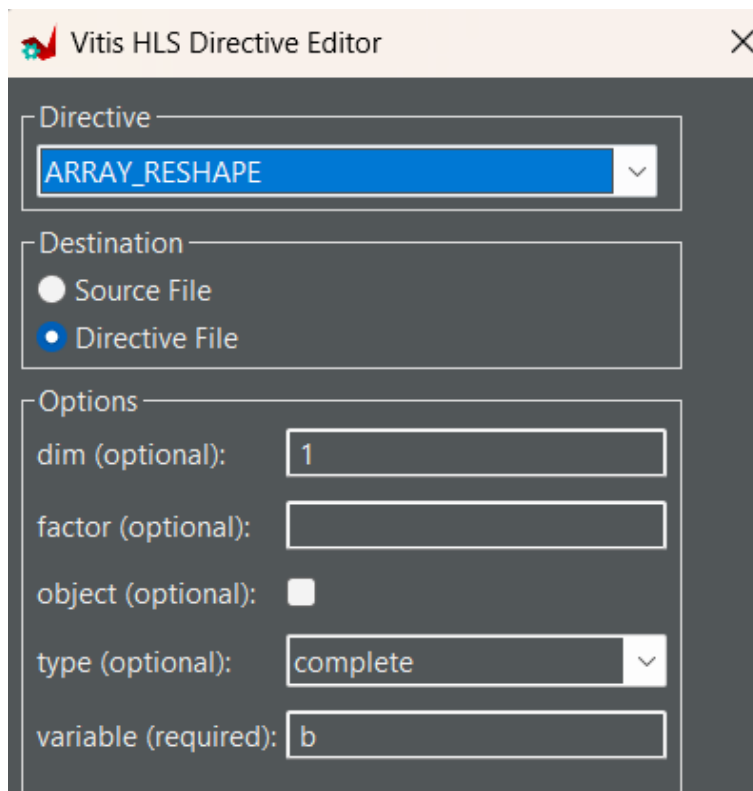


Hình 41 Tab Directive cho solution4

Tại a và b bấm chuột phải chọn Insert Directive chọn ARRAY_RESHAPE

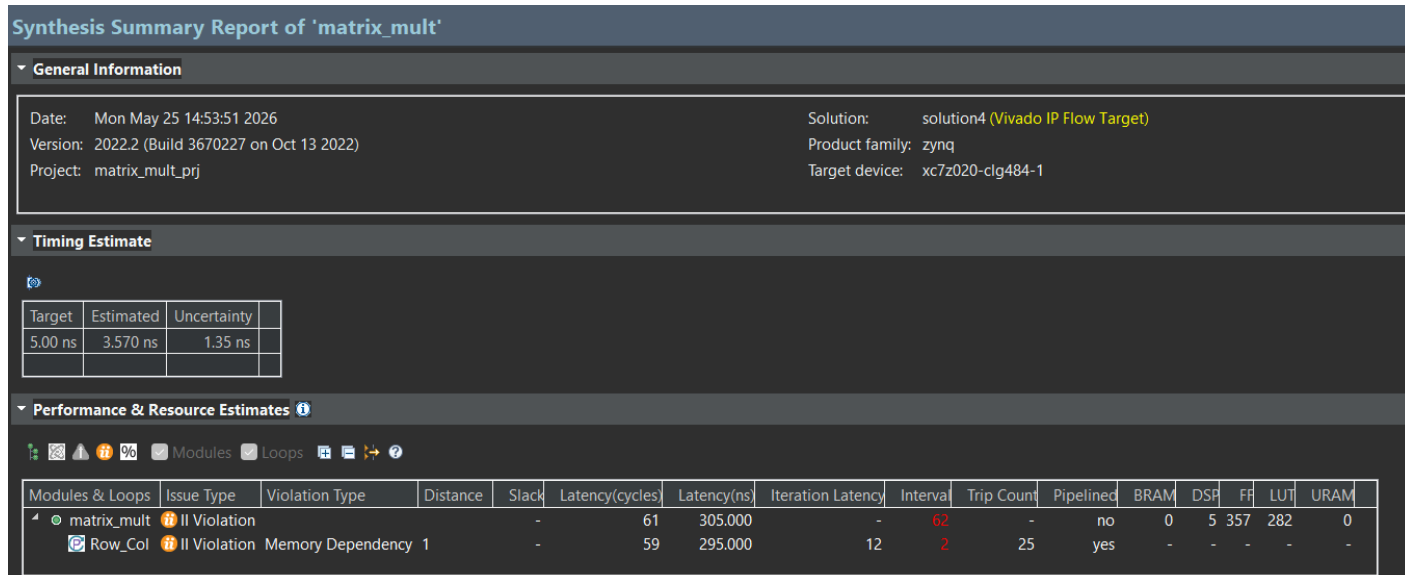


Hình 42 Cửa sổ cấu hình cách tối ưu cho a



Hình 43 Cửa sổ cấu hình cách tối ưu cho b

Solution4 này chọn ARRAY_RESHAPE để yêu cầu trình biên dịch tổ chức lại cách lưu trữ dữ liệu của mảng trên bộ nhớ vật lý, mà không làm thay đổi bất kỳ dòng code



Hình 44 Cửa sổ synthesis của solution4

Chọn Run C synthesis để mở cửa sổ synthesis summary của solution4.

- Time Estimate (Ước lượng thời gian): Chu kỳ xung nhịp ước lượng đạt 3.570 ns, nhỏ hơn so với chu kỳ mục tiêu 5.00 ns, chứng tỏ thiết kế đáp ứng yêu cầu timing. Với chu kỳ 3.570 ns, thiết kế về mặt lý thuyết có thể đạt tần số khoảng 280 Ghz, cao hơn tần số mục tiêu 200 Ghz. Vì vậy, solution4 đạt timing tốt và có khả năng hoạt động ổn định ở clock yêu cầu.

- Độ trễ (Latency): Toàn bộ hàm matrix_mult có latency 61 chu kỳ clock, tương ứng khoảng 305 ns. Điều này có nghĩa là từ lúc bắt đầu đưa dữ liệu đầu vào cho đến khi hoàn thành phép nhân ma trận và tạo ra kết quả đầu ra, thiết kế cần khoảng 61 chu kỳ clock. So với các solution có latency lớn hơn, solution4 cho thấy tốc độ xử lý đã được cải thiện đáng kể.

- Khoảng cách khởi tạo (Interval - II): Toàn hàm matrix_mult có interval là 62 chu kỳ, nghĩa là sau khoảng 62 chu kỳ hệ thống mới có thể bắt đầu xử lý một bộ dữ liệu mới. Riêng vòng lặp Row_Col đã được pipeline và có II = 2, tức là sau mỗi 2 chu kỳ clock, pipeline mới có thể bắt đầu một vòng lặp mới. Báo cáo xuất hiện cảnh báo màu cam II Violation do Memory Dependency, cho thấy công cụ HLS đã cố gắng đạt II nhỏ hơn nhưng không thể đạt II = 1 do tồn tại phụ thuộc dữ liệu hoặc giới hạn truy cập bộ nhớ.

- Vòng lặp Row_Col: Loop Row_Col có latency 59 chu kỳ, iteration latency 12, trip count 25 và được đánh dấu pipelined = yes. Điều này cho thấy vòng lặp chính đã được pipeline, giúp tăng khả năng xử lý song song giữa các lần lặp. Tuy nhiên, do vẫn còn memory dependency nên hiệu quả pipeline chưa đạt mức tối ưu nhất.

- Resource Utilization (Sử dụng tài nguyên): Thiết kế sử dụng 0 BRAM, 5 DSP, 357 FF và 282 LUT. Việc dùng 5 DSP cho thấy các phép nhân trong ma trận đã được triển khai bằng khối DSP chuyên dụng, giúp tăng tốc tính toán. FF dùng để lưu dữ liệu và trạng thái qua các chu kỳ clock, còn LUT dùng cho logic điều khiển và các phép toán tổ hợp. Thiết kế không dùng BRAM, cho thấy dữ liệu mảng không cần ánh xạ vào bộ nhớ khối lớn hoặc đã được tổ chức bằng tài nguyên logic/thanh ghi.

- Nhận xét tổng quát: Solution4 đạt timing với chu kỳ ước lượng 3.570 ns, nhỏ hơn mục tiêu 5 ns. Latency chỉ còn 61 cycles, thấp hơn nhiều so với solution2, cho thấy hiệu năng đã được cải thiện rõ rệt. Tuy nhiên, cảnh báo II Violation do Memory Dependency vẫn còn tồn tại, khiến vòng lặp Row_Col chỉ đạt II = 2 thay vì II = 1. Nhìn chung, solution4 là một phương án tối ưu khá tốt vì cân bằng được giữa tốc độ xử lý và tài nguyên phần cứng, sử dụng tài nguyên không quá cao nhưng giảm latency đáng kể.

Fail Fast

Created on Fri May 29 08:40:41 +0700 2026 with report_failfast (2021.05.07)

Design Summary

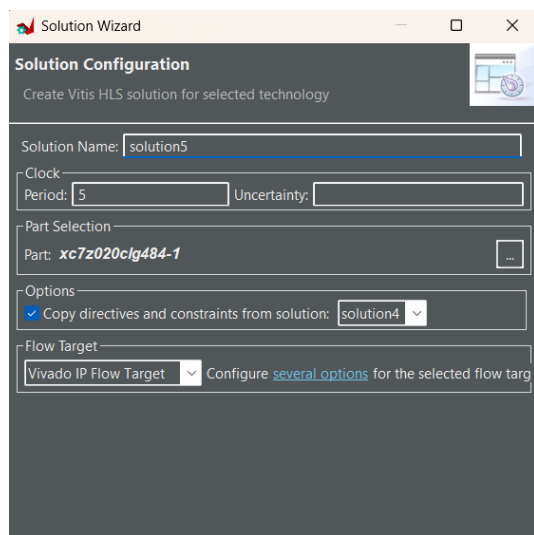
design_1

xc7z020clg484-1

Criteria	Guideline	Actual	Status
LUT	70%	0.18%	OK
FD	50%	0.16%	OK
LUTRAM+SRL	25%	0.03%	OK
MUXF7	15%	0.00%	OK
DSP	80%	2.27%	OK
RAMB/FIFO	80%	0.00%	OK
DSP+RAMB+URAM (Avg)	70%	2.27%	OK
BUFGCE* + BUFGCTRL	24	0	OK
DONT_TOUCH (cells/nets)	0	0	OK
MARK_DEBUG (nets)	0	0	OK
Control Sets	998	12	OK
Average Fanout for modules > 100k cells	4	0	OK
Non-FD high fanout nets > 10k loads	0	0	OK
TIMING-6 (No common primary clock between related clocks)	0	0	OK
TIMING-7 (No common node between related clocks)	0	0	OK
TIMING-8 (No common period between related clocks)	0	0	OK
TIMING-14 (LUT on the clock tree)	0	0	OK
TIMING-35 (No common node in paths with the same clock)	0	0	OK
Number of paths above max LUT budgeting (0.575ns)	0	0	OK
Number of paths above max Net budgeting (0.403ns)	0	0	OK

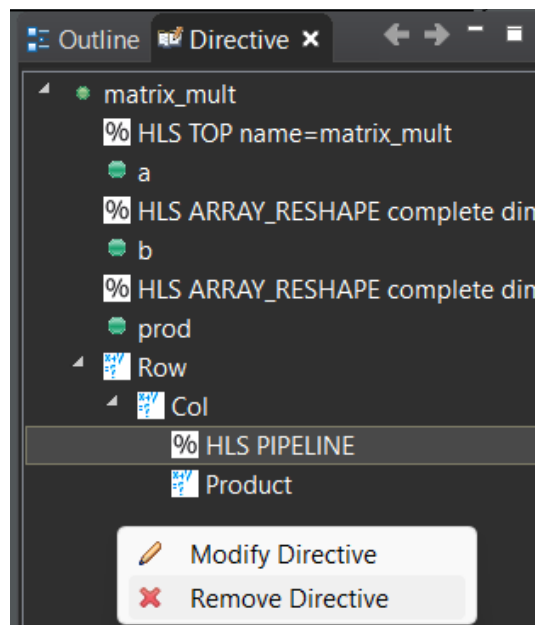
Hình 45 Implementation của solution 4

Báo cáo Fail Fast cho thấy thiết kế đạt tất cả các tiêu chí kiểm tra nhanh với trạng thái OK. Mức sử dụng tài nguyên rất thấp, trong đó LUT chỉ chiếm 0.18%, FF chiếm 0.16%, DSP chiếm 2.27% và không sử dụng BRAM/FIFO. Không có lỗi về clock, fanout, timing path hoặc net delay. Vì vậy, thiết kế có kích thước nhỏ, ổn định, không gây áp lực lên tài nguyên FPGA và có khả năng tiếp tục implementation/bitstream mà không gặp vấn đề nghiêm trọng.



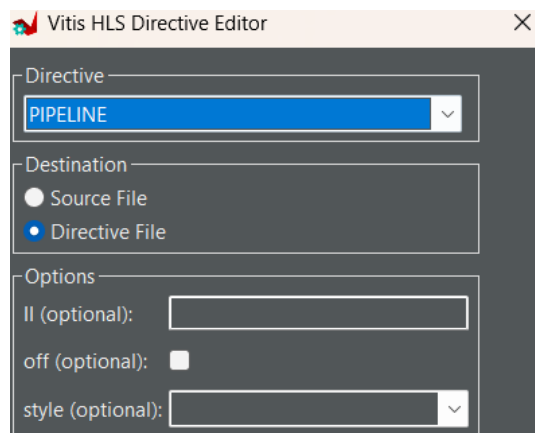
Hình 46 Cửa sổ tạo solution5

Tạo solution5 như solution trước

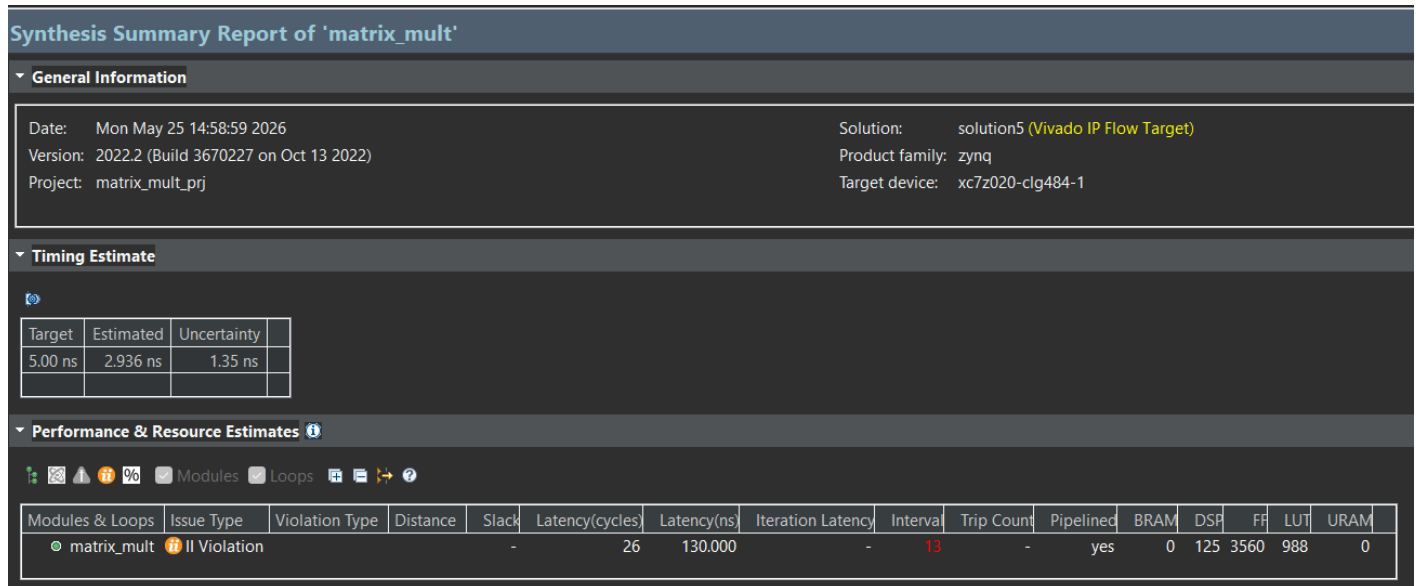


Hình 47 Tab Directive của solution5

Nhấn chuột phải vào chỗ trống chọn Remove Directive gỡ bỏ lệnh cấu hình tối ưu
 Thêm Directive PIPELINE vào mức cao nhất matrix_mult



Hình 48 Cửa sổ cấu hình cho matrix_mult



Hình 49 Cửa sổ synthesis solution5

- Time estimate: Chu kỳ clock ước lượng đạt 2.936 ns đạt yêu cầu dưới 5 ns được đặt ra cho thấy thiết kế mà quá trình synthesis đạt được có tốc độ cao nhất trong tất cả các solution đã thử trước đó. Thiết kế này có thể hoạt động ổn định ở tần số tối đa lên đến 340.6 GHz.
- Độ trễ: 130 ns tương ứng với 26 chu kỳ clock. Thiết kế cần tiêu tốn 26 chu kỳ hoạt động tính để thực hiện và đưa ra kết quả cho một phép tính. Đây cũng là kết quả tốt nhất trong tất cả 5 solution.
- II: Các vòng lặp được flatten xuống chỉ còn matrix_mult thì vì Row_col như trước đó. Với số Interval = 13, pipeline chưa thể hoạt động ở mức độ tối ưu là Interval = 1, cần phải sau 13 chu kỳ clk thì pipeline mới có thể bắt đầu một interval mới vì cần phải thực hiện các thao tác đọc, ghi, lặp lại các phép tính nhân, cộng và độ trễ sinh ra trong quá trình tính toán.
- Resource Utilization: Đây là solution sử dụng nhiều tài nguyên nhất. Thiết kế sử dụng 0 BRAM, 125 DSP, 3563 FF, 988 LUT và 0 URAM, cao hơn nhiều lần so với các solution trước đặc biệt với DSP và FF vì quá trình xử lý được tối ưu tốt hơn cho việc xử lý pipeline nên số lượng dữ liệu cần lưu trữ và sử dụng tăng dẫn đến số lượng FF tăng nhiều. DSP cũng tăng nhiều vì các khối DSP được thiết kế để thực hiện việc nhân chập nên HLS sử dụng thêm nhiều DSP để thực hiện phép toán nhân chập song song với nhau thay vì như ở solution 1 chỉ sử dụng 5 DSP vì chưa áp dụng pipeline hay ở các solution khác khi pipeline được áp dụng ở các vòng lặp con.

Cosimulation Report for 'matrix_mult'

General Information

Date: Fri May 29 08:45:22 SEAST 2026

Version: 2023.2 (Build 4023990 on Oct 11 2023)

Project: matrix_mult_prj

Status: Pass

Solution: solution5 (Vivado IP Flow Target)

Product family: zynq

Target device: xc7z020-clg484-1

Cosim Options

Tool: Vivado XSIM

RTL: Verilog

Performance Estimates

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency	Total Execution Time
matrix_mult				26	26	26	26

Hình 50 Cosimulation solution 5

Trạng thái Status: Pass cho thấy bộ mã hóa phân cứng RTL hoạt động chính xác về mặt chức năng được mô tả trong source C. Kết quả của quá trình Co-simulation cho thấy Latency = 26, mức độ trễ mà phần mềm đo được trong quá trình C-simulation 26 tương đương với kết quả synthesis. Từ kết quả Co-simulation, có thể thấy quá trình tạo code RTL thành công.

Criteria	Guideline	Actual	Status
LUT	70%	1.19%	OK
FD	50%	1.58%	OK
LUTRAM+SRL	25%	0.00%	OK
MUXF7	15%	0.00%	OK
DSP	80%	55.91%	OK
RAMB/FIFO	80%	0.00%	OK
DSP+RAMB+URAM (Avg)	70%	55.91%	OK
BUFGCE* + BUFGCTRL	24	0	OK
DONT_TOUCH (cells/nets)	0	0	OK
MARK_DEBUG (nets)	0	0	OK
Control Sets	998	28	OK
Average Fanout for modules > 100k cells	4	2.00	OK
Max Average Fanout for modules > 100k cells	4	0	OK
Non-FD high fanout nets > 10k loads	0	0	OK
TIMING-6 (No common primary clock between related clocks)	0	0	OK
TIMING-7 (No common node between related clocks)	0	0	OK
TIMING-8 (No common period between related clocks)	0	0	OK
TIMING-14 (LUT on the clock tree)	0	0	OK
TIMING-35 (No common node in paths with the same clock)	0	0	OK
Number of paths above max LUT budgeting (0.575ns)	0	0	OK
Number of paths above max Net budgeting (0.403ns)	0	0	OK

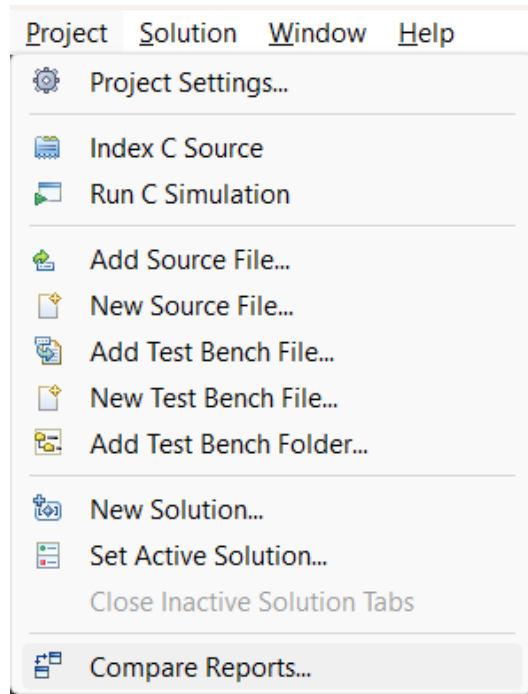
Hình 51 Phân bố implementation trên FPGA solution 5

- Resource Usage: Mức sử dụng thực tế (Actual) của một số tài nguyên trong thiết kế thấp hơn rất nhiều so với Guideline trong khi một vài tài nguyên lại sử dụng nhiều như LUT chỉ dùng 1.19%, Guideline là 70% trong khi DSP dùng đến 55.91%, Guideline 80%.

- Timing: Không phát hiện bất kỳ cấu trúc thiết kế rủi ro hay vi phạm luật đi dây xung nhịp nào (tổng các dòng TIMING đều là 0).
- Status: Tất cả các hạng mục đều báo OK.

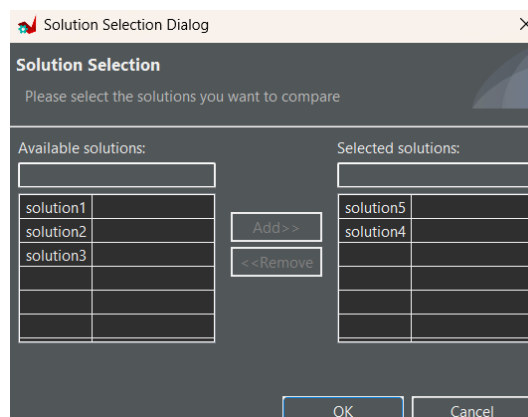
Kết luận: Tuy thiết kế được ước lượng có tốc độ nhanh nhất trong tất cả 5 solution nhưng trong Implementation thực tế kết quả đạt được lại thấp hơn kết quả ước lượng ở Synthesis trong khi sử dụng nhiều tài nguyên hơn các thiết kế khác. Có thể cần phải điều chỉnh lại solution này để đạt được một kết quả tốt hơn trong Implementation.

Chọn Run C Synthesis để mở cửa sổ synthesis summary solution5



Hình 52 Mục Compare Report

Chọn Project → Compare Reports để so sánh các thông số như Latency, Utilization giữa solution4 và solution5



Hình 53 Cửa sổ lựa chọn các solution so sánh với nhau

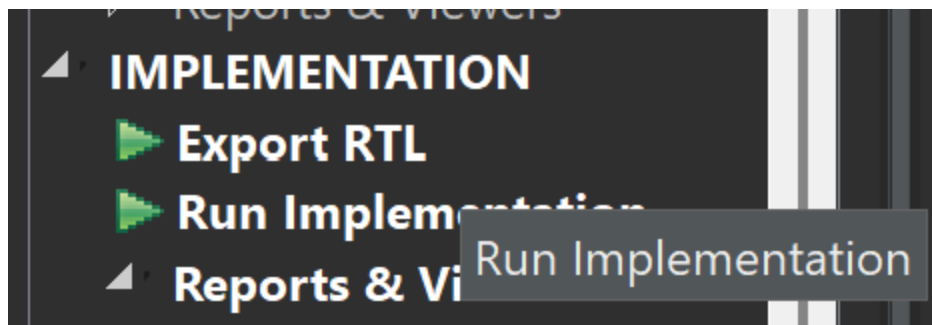
Performance Estimates			
Timing			
Clock		solution5	solution4
ap_clk	Target	5.00 ns	5.00 ns
	Estimated	2.936 ns	3.570 ns
Latency			
		solution5	solution4
Latency (cycles)	min	26	61
	max	26	61
Latency (absolute)	min	0.130 us	0.305 us
	max	0.130 us	0.305 us
Interval (cycles)	min	13	62
	max	13	62
Utilization Estimates			
		solution5	solution4
BRAM_18K	0	0	
DSP	125	5	
FF	3560	357	
LUT	988	282	
URAM	0	0	

Hình 54 Cửa sổ so sánh thông số giữa solution4 và solution5

Thông số	Solution 4	Solution 5	Đánh giá
Target Clock	5.00 ns	5.00 ns	Cả hai đều đặt cùng mục tiêu timing
Estimated Clock	3.570 ns	2.936 ns	Solution5 nhanh hơn
Latency (cycles)	61 cycles	26 cycles	Solution5 xử lý nhanh hơn nhiều
Latency (absolute)	0.305 μ s	0.130 μ s	Solution5 có thời gian xử lý nhỏ hơn
Interval (II)	62 cycles	13 cycles	Solution5 có throughput tốt hơn
BRAM_18K	0	0	Không dùng BRAM
DSP	5	125	Solution5 dùng nhiều DSP hơn
FF	357	3560	Solution5 tốn nhiều Flip-Flop hơn
LUT	282	988	Solution5 dùng nhiều LUT hơn
URAM	0	0	Không dùng URAM

Kết luận:

Run implementation



Hình 55 Chọn Run Implementation

Sau khi Run Implementation từng solution thì ta được bảng như sau:

Bảng so sánh resource usage và final timing giữa các solution

Solution	LUT	FF	DSP	BRAM	SRL	CP required	CP achieved post-synthesis	Timing
solution1	104	161	5	0	0	5.000 ns	3.191 ns	Met
solution2	89	117	1	0	5	5.000 ns	4.228 ns	Met
solution3	104	161	5	0	0	5.000 ns	3.191 ns	Met
solution4	94	167	5	0	6	5.000 ns	3.089 ns	Met
solution5	633	1676	123	0	0	5.000 ns	3.884 ns	Met

Nhận xét: Qua bảng so sánh, có thể thấy các solution đều đạt timing với chu kỳ yêu cầu 5 ns.

Solution2 sử dụng ít tài nguyên nhất, phù hợp khi muốn tiết kiệm phần cứng.

Solution4 cho kết quả timing tốt nhất với CP achieved 3.089 ns, trong khi tài nguyên chỉ tăng nhẹ.

Solution5 sử dụng nhiều tài nguyên nhất, đặc biệt là DSP và FF, cho thấy các kỹ thuật tối ưu mạnh như pipeline/unroll có thể cải thiện hiệu năng nhưng làm tăng đáng kể chi phí phần cứng.

Exercise 3C Interface Synthesis

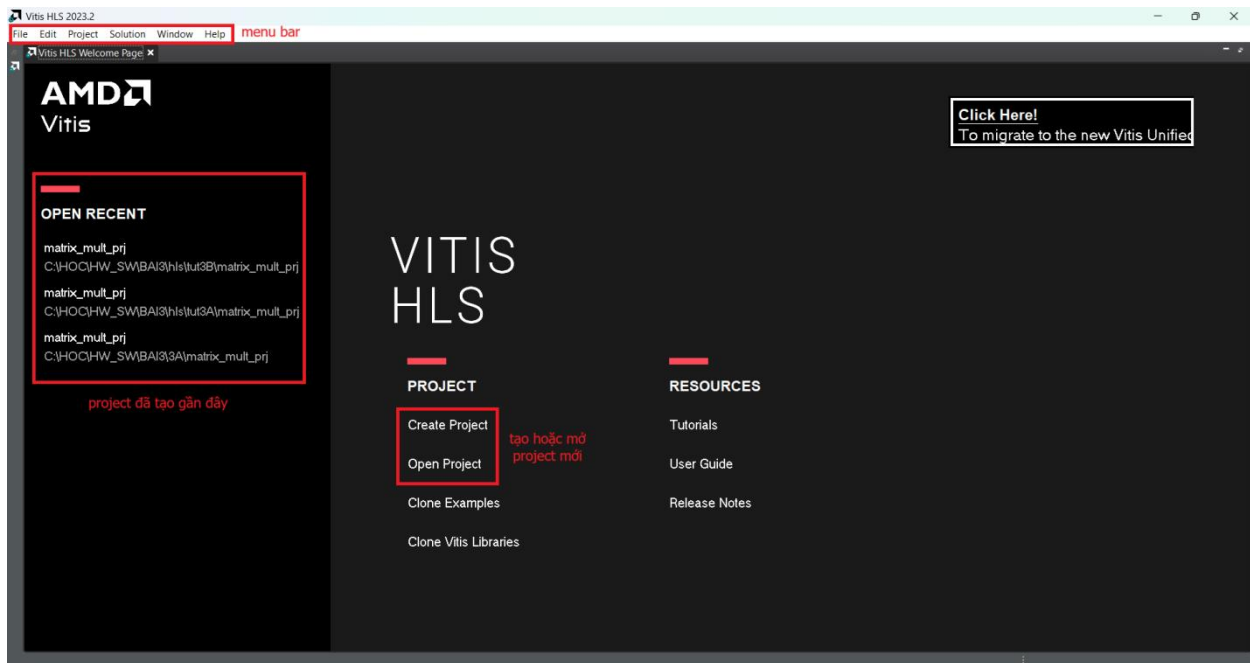
Bước 1: Tạo project từ source có sẵn

a. Trước khi tạo project, ta cần phải có cái file source để tạo project này. Theo file tutorial, cần phải có Zynq_Book\sources\hls chứa source của bài tutorial 3. Các file này có thể được tìm thấy trên trang GitHub sau: <https://github.com/RayHightower/zynq-book>. Tiến hành tải repo này xuống để sử dụng cho bài này.

b. Để tiến hành tạo project, có thể sử dụng cách làm bằng GUI Vitis HLS hoặc sử dụng command prompt của Vitis HLS như sau:

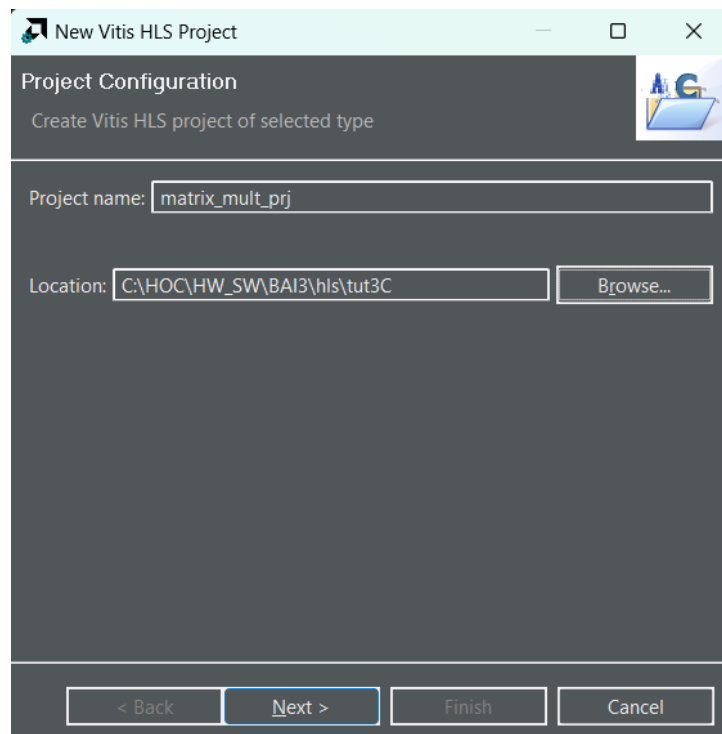
Cách GUI:

- Mở Vitis HLS trên máy tính, sau khi mở chương trình có giao diện như hình



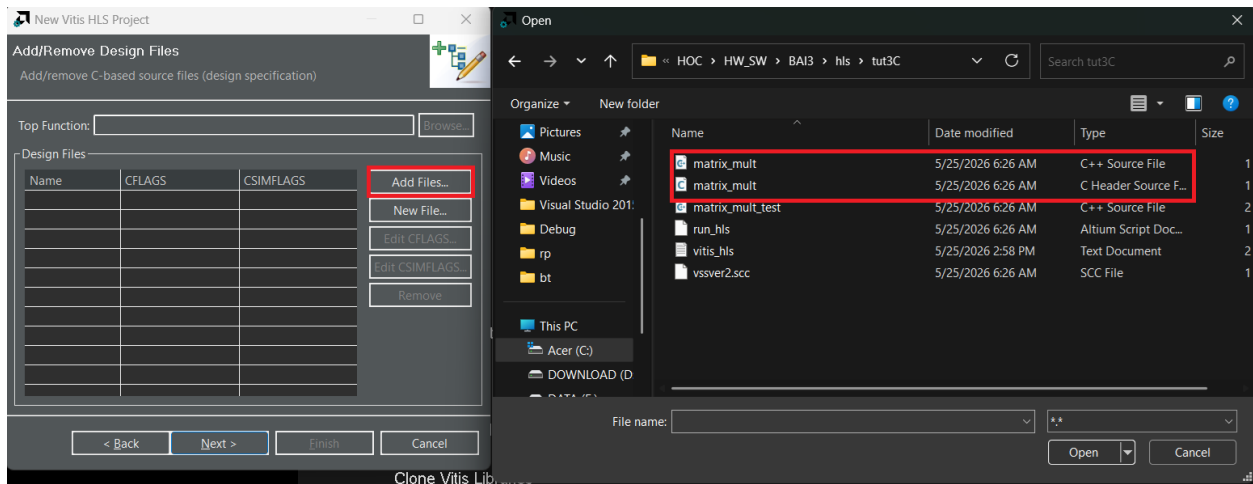
Hình 56 Giao diện GUI Vitis HLS

- Sau khi đã mở Vitis HLS, tạo một project mới bằng cách chọn “Create Project” trên Welcome Page hoặc chọn File > New Project.
- Tiến hành đặt tên và chọn đường dẫn cho project rồi nhấn “Next”. Có thể đặt tên project như trong tutorial (matrix_mult_prj).



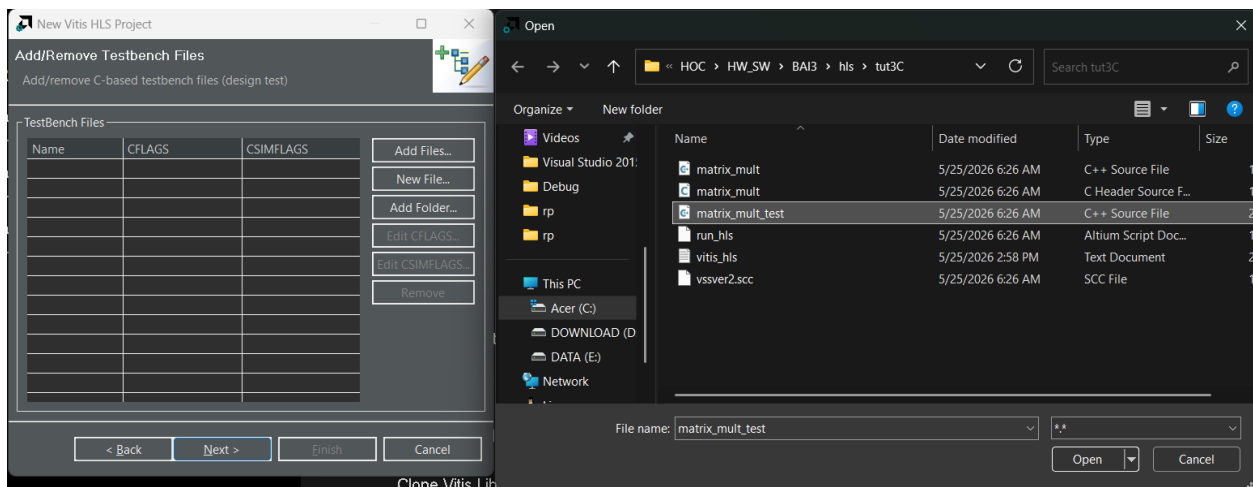
Hình 57 Hộp thoại tạo project

- Tiếp theo, cần phải thêm các file source cpp đã được cài đặt ở phần a. Đây là file cho bộ nhân ma trận được sử dụng như một thư viện trong file testbench. Chọn “Add Files...” rồi thêm các file matrix_mult.cpp và file header của nó như hình 3 rồi chọn “Next”.



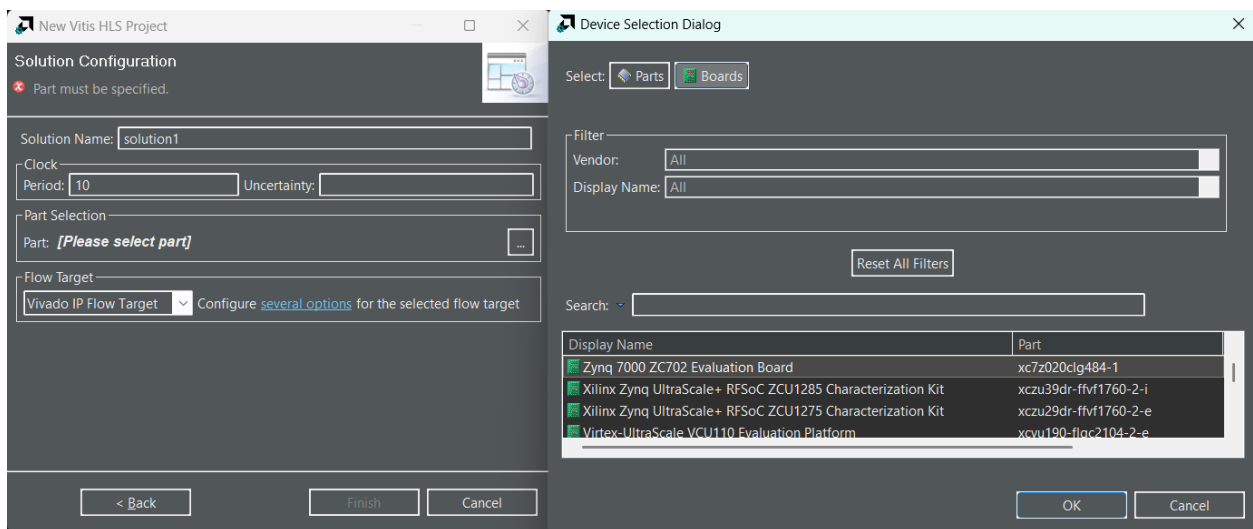
Hình 58: Thêm file source C cho project

- Sau đó, cần phải thêm file testbench là file dùng để kiểm tra và đánh giá thiết kế này. Chọn “Add Files...” rồi thêm các file matrix_mult_test.cpp như hình 4 rồi chọn “Next”.



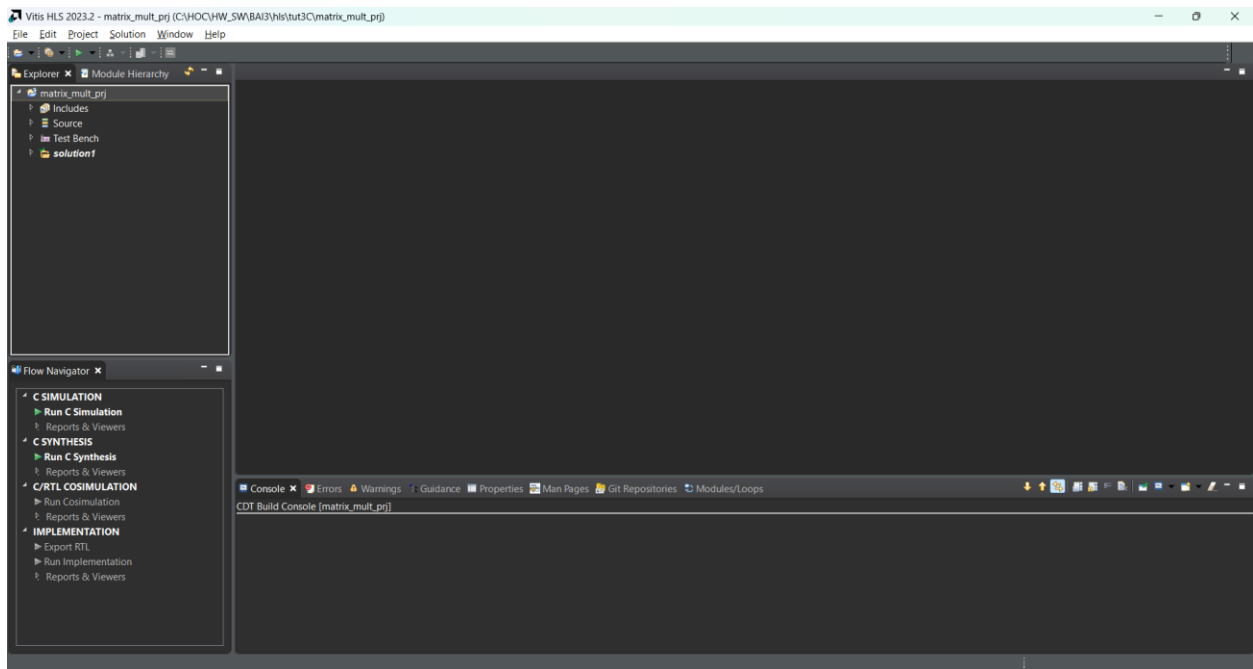
Hình 59 Thêm file testbench cho project

- Cuối cùng là tạo solution để chọn và lưu cấu hình của hệ thống, đặt Period là 5 và để Uncertainty trống. Sau đó, chọn board cho project ở phần Part Selection.



Hình 60 Tạo solution và chọn board

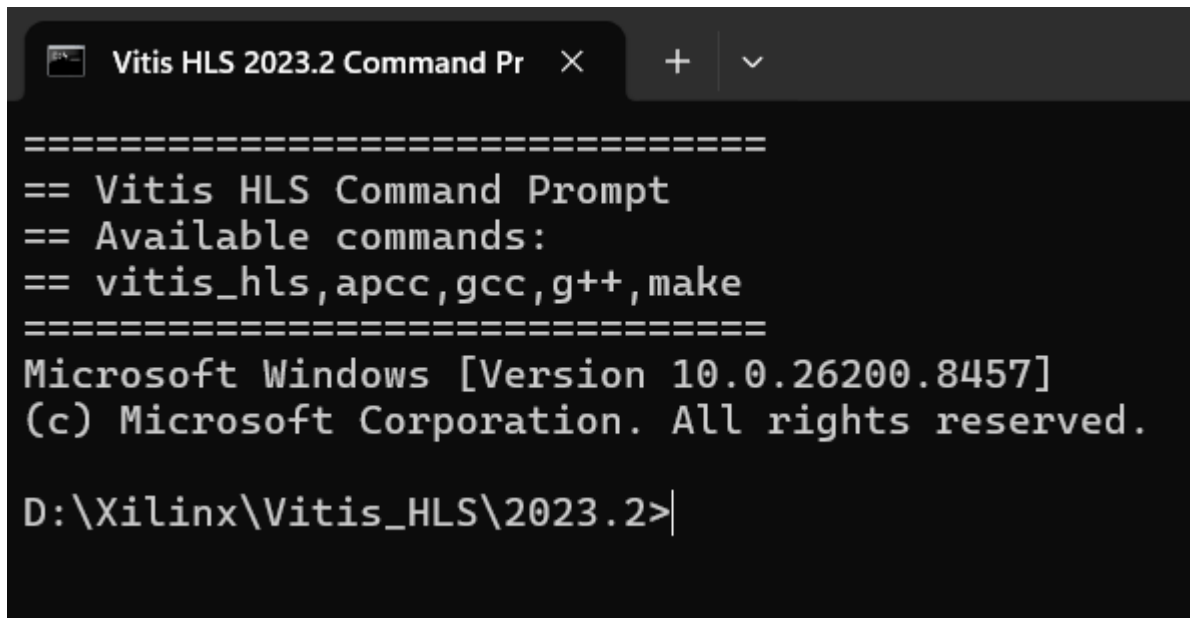
- Sau khi đã cấu hình cho project xong, giao diện GUI Vitis HLS sẽ mở lên như hình 6:



Hình 61 Giao diện GUI của Vitis HLS

Cách command prompt:

- Mở Vitis HLS command prompt trên máy tính, sau khi mở chương trình có giao diện như hình sau:



Hình 62 Giao diện Vitis HLS Command Prompt

- Tiến hành sử dụng các lệnh cd để chuyển đến folder hls\tut3C chứa các file source cho project, sử dụng lệnh “<ô đĩa>:” (ví dụ “C:”) để chỉ đến ổ đĩa hoặc phân vùng chứa folder cần tìm.
- Tiếp theo, tiến hành tạo project với lệnh “vitis_hls -f run_hls.tcl” (thay vitis_hls với vivado_hls nếu dùng bản cũ). Sau khi đợi HLS chạy xong, nếu thành công, trên command prompt sẽ xuất hiện:

Test passed!

INFO: [SIM 211-1] CSim done with 0 errors.

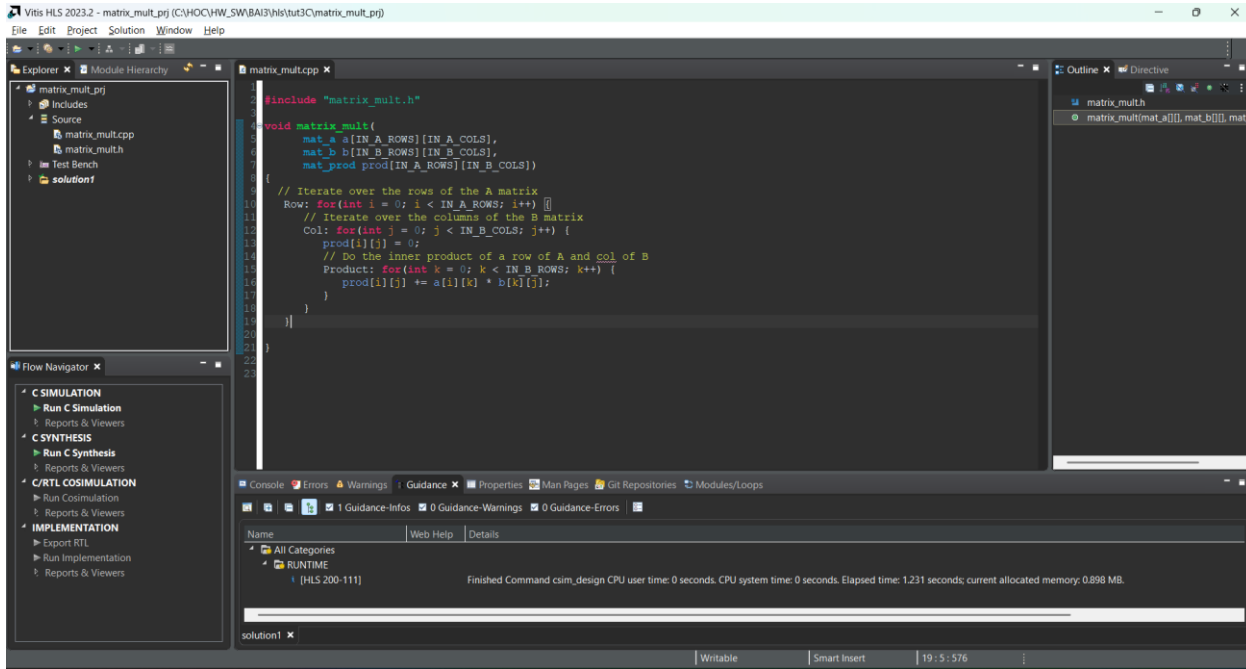
INFO: [SIM 211-3] ***** CSIM finish *****

INFO: [HLS 200-111] Finished Command csim_design CPU user time: 0 seconds. CPU system time: 0 seconds. Elapsed time: 1.231 seconds; current allocated memory: 0.898 MB.

INFO: [HLS 200-112] Total CPU user time: 1 seconds. Total CPU system time: 1 seconds. Total elapsed time: 3.168 seconds; peak allocated memory: 103.473 MB.

INFO: [Common 17-206] Exiting vitis_hls at Mon May 25 14:53:05 2026...

- Sau đó, ta có thể mở GUI lên với lệnh “vitis_hls -p matrix_mult_prj” (thay vitis_hls với vivado_hls nếu dùng bản cũ). Giao diện GUI Vitis HLS sẽ mở lên như hình 8:



Hình 63 Giao diện GUI Vitis HLS

Bước 2: Chạy C synthesis để tổng hợp source C sang source RTL và đánh giá kết quả

a. Từ trong Tab Explorer, chọn matrix_mult_prj/Source/matrix_mult.cpp để mở file source lên rồi tìm đến tab Flow Navigator và chọn “Run C Synthesis” để bắt đầu chạy synthesis. Quá trình này tương tự như ở bài tutorial 3B và kết quả cũng tương tự. Sau khi chạy xong, mở file report bằng cách chọn Reports & Viewers/Report rồi kéo xuống phần “HW Interfaces”. Đây là các tín hiệu giao diện của phần cứng gồm tên port, hướng (in/out) và độ rộng port (theo bit) và các tín hiệu điều khiển sẽ có tên, loại (clock, reset,...) và port.

▼ HW Interfaces			
▼ AP_MEMORY			
Port	Direction	Bitwidth	
a_address0	out	5	
a_address1	out	5	
a_q0	in	8	
a_q1	in	8	
b_address0	out	5	
b_address1	out	5	
b_q0	in	8	
b_q1	in	8	
prod_address0	out	5	
prod_d0	out	16	
▼ TOP LEVEL CONTROL			
Interface	Type	Ports	
ap_clk	clock	ap_clk	
ap_rst	reset	ap_rst	
ap_ctrl	ap_ctrl_hs	ap_done ap_idle ap_ready ap_start	

Hình 64 Report HW Interface cho solution1

Từ kết quả HW Interfaces trong report, có thể thấy HLS sử dụng protocol ap_memory cho phép nhân ma trận. Đây là vì trong source cpp, việc lựa chọn kiểu dữ liệu cho ma trận là mảng tương đương với cấu trúc của dữ liệu, có thể truy cập dữ liệu này với địa chỉ và lấy được giá trị của 2 ma trận a, b và ma trận là kết quả của phép nhân là prod cũng là một mảng.

Nếu quan sát file testbench, có thể thấy 2 ma trận cần nhân là 2 ma trận 5x5 và trong file .h, các kiểu “mat_a”, “mat_b” là char 1 byte và “mat_prod” là short 2 byte tức là để bộ nhớ chứa 1 mảng input là a hoặc b sẽ cần 25 byte (5x5x1 byte) và 50 byte (5x5x2 byte) cho ma trận output là prod. Vì vậy port dữ liệu input mảng a hoặc b có bitwidth 8b để đủ chứa kiểu dữ liệu char và đường địa chỉ 5b để chứa có đủ $2^5 = 32$ giá trị cho mảng 5x5. Mảng prod cũng như vậy nhưng đường dữ liệu là 16b cho kiểu dữ liệu short.

Ngoài các đường dữ liệu, HLS cũng tạo ra các tín hiệu điều khiển với protocol Top Level Control giúp tạo ra các tín hiệu cần thiết để điều khiển khối phần cứng này. Tín hiệu ap_clk là tín hiệu clock để đồng bộ hoạt động của khối, ap_rst để reset trạng thái của khối. Ngoài ra, còn có ap_ctrl là giao thức handshaking ap_ctrl_hs gồm 4 port sau:

- ap_start: Ra lệnh cho khối bắt đầu xử lý
- ap_done: Khối tạo tín hiệu khi đã xử lý xong và có dữ liệu sẵn sàng

- ap_idle: Khởi tạo tín hiệu khi không thực hiện tác vụ nào
- ap_ready: Khởi tạo tín hiệu sẵn sàng nhận dữ liệu mới

HLS chọn ap_ctrl_hs vì hàm matrix_mult() return void nên handshaking được chọn để quản lý luồng hoạt động của khối.

Hình nhóm:

Điểm danh từ trái qua phải: Tuấn Kiệt – Thái Khương – Phi Hùng - Thành Đô – Quang Huy (thứ 2)



Điểm danh: Phi Hùng – Thái Khương – Tuấn Kiệt – Thành Đô – Quang Huy (thứ 6)



LINK GITHUB

https://github.com/LDTKhuong/LAB1_Nhom_15